



Efficient matheuristics to solve a rich production-routing problem

Alexandre Fortes^{a,*}, Ricardo Camargo^b, Leandro Reis Muniz^a, Fátima Machado de Souza Lima^c,
Fernanda dos Reis Cota^d

^a Departamento de Engenharia Mecânica e Produção, Universidade Federal de São João del-Rei, Centro, São João del-Rei, Minas Gerais, Brazil

^b Departamento de Engenharia de Produção, Universidade Federal de Minas Gerais, Pampulha, Belo Horizonte, Minas Gerais, Brazil

^c Departamento de Ciências Administrativas, Universidade Federal de Minas Gerais, Pampulha, Belo Horizonte, Minas Gerais, Brazil

^d Departamento de Engenharia de Produção, Administração e Economia, Universidade Federal de Ouro Preto, Bauxita, Ouro Preto, Minas Gerais, Brazil

ARTICLE INFO

Keywords:

Production-routing problem
Iterated local search
Hybrid methods
Matheuristics

ABSTRACT

We present a rich production-routing problem having limited production and storage capacities at the plant, limited storage capacity at the clients, a heterogeneous fleet subjected to a maximum riding time, and allowing for back-orders to meet unfulfilled demands at penalty cost. As the problem scales quickly with the number of customers, periods, products, and vehicles, three hybrid two-level decomposition approaches using a top-down strategy were devised. The top tier determines the production and inventory levels, and the distribution of goods via CPLEX, that is, it makes tactical decisions, while the bottom tier heuristically routes a heterogeneous fleet in each period, that is, it makes operational decisions. The proposed methods rely on an iterated local search framework that combines tailored perturbation schemes prioritizing either tactical or operational decisions, or both. The main new feature of the algorithms is the adoption of an implicit cost that estimates the delivery routing costs when making production, holding, and transportation decisions. This implicit cost serves as an important guide to obtain improved solutions. The algorithms were tested over an extensive set of instances, and the results demonstrated that all methods overcome CPLEX by obtaining more, better, and faster solutions with much less computational effort. The devised heuristic, which prioritizes operational-level decisions during the perturbation phase, attained the best overall results.

1. Introduction

The inventory-routing problem (IRP) and its many variants combine inventory management and vehicle-routing decisions to minimize costs while improving supply chain performance and customer service levels. When production decisions are also incorporated during modeling, it gives rise to a more general problem known as the production-routing problem (PRP), thereby enabling further savings. The savings achieved by combining such decisions into a single problem have already been discussed by several authors addressing different practical applications, e.g., Absi, Archetti, Dauzère-Pérès, Feillet, and Speranza (2018), Archetti and Speranza (2016), Brown, Keegan, Vigus, and Wood (2001), Çetinkaya, Üster, Easwaran, and Keskin (2009), Côté, Guastaroba, and Speranza (2017) and Stålhane, Andersson, Christiansen, and Fagerholt (2014), being Chandra (1993) one of the early works to point them out.

Most of these applications are within the context of vendor-managed inventory systems wherein a supplier has to efficiently manage retailers' stock levels by deciding the time and product quantities to deliver to the retailers so that stockouts do not occur.

Stockout is a situation wherein an item is out of stock either by defective shelf replenishment practices or surges in the retailers' demands that were not properly foreseen or expected by the supplier. However, as highlighted by Brahimi and Aouam (2016), a supply chain disruption due to limited production or depot storage capacities, fleet-related problems that cause delays in replenishment, or, as the whole world is now enduring, the outbreak of a pandemic.

Furthermore, there are many practical cases, especially in the fast-moving consumer goods (FMCG) industry, where stockouts are dealt on a regular basis, forcing managers to handle back-orders, that is, managers have to satisfy a retailer's demand for a given period on subsequent periods, on a regular basis.

FMCG are goods having a useful life shorter than a year and are bought fairly frequently with recurring expenditures. Examples of such products are food, beverages, personal care, household and cleaning products, apparel, and footwear. Despite its large market size, the US FMCG market size alone was \$635 billion in 2019, and despite the rich literature on PRP (Adulyasak et al., 2015b), there are only a

* Corresponding author.

E-mail addresses: afortes@ufsj.edu.br (A. Fortes), rcamargo@dep.ufmg.br (R. Camargo), leandro.reis@ufsj.edu.br (L.R. Muniz), fatimamslima@ufmg.br (F.M.S. Lima), fernanda.cota@aluna.ufop.edu.br (F.R. Cota).

<https://doi.org/10.1016/j.cie.2022.108369>

Received 10 November 2021; Received in revised form 2 May 2022; Accepted 21 June 2022

Available online 25 June 2022

0360-8352/© 2022 Elsevier Ltd. All rights reserved.

few studies that have addressed back-ordering in the context of PRP, namely [Brahimi and Aouam \(2016\)](#) and [Bayley and Bookbinder \(2019\)](#).

[Brahimi and Aouam \(2016\)](#) proposed two mixed-integer linear programs for a rich PRP with a production facility with limited production and storage capacities, considering multiple products delivered by a homogeneous fleet to geographically dispersed clients that also have limited storage capacities, while allowing for back-ordering at a penalty cost. They proposed a relax-and-fix heuristic combined with a local search approach to solve their models and computationally assessed the impact and effects of different configurations on the production-storage capacities as well as the demand patterns on back-ordering. The results highlighted the importance of having back-orders when addressing capacitated systems. Finally, [Bayley and Bookbinder \(2019\)](#) also proposed a branch-and-cut method for rich PRP with back-orders but with no clear description of their algorithm and the generated instances. Here, we present a rich variant of PRP comprising determining at minimal total cost, a production and distribution plan that fulfills periodic demands of multiple products by different clients while controlling the inventory levels at the production plant and clients, and satisfying production and storage capacities, that allows for back-orders for the unfilled demands in a timely fashion and assumes a heterogeneous capacitated fleet with a maximum riding time to deliver the products in each period of the planning horizon. The total cost is made up of production and distribution-related costs. The former includes the production, setup, holding, and inventory costs, while the latter accounts for vehicle fixed and routing costs.

To solve this new variant, we designed three efficient hybrid iterated local search matheuristics that decompose the problem into two smaller subproblems: one, a tactical problem that decides what to make and to deliver at each period, while satisfying the production and storage capacities, and the other, an operational problem that actually plans routes satisfying the maximum riding time of the fleet. While the former subproblem is solved exactly with CPLEX, the latter is heuristically tackled. Tailored perturbation operators are devised and applied to both subproblems to escape from non-promising solutions.

Although the top-down solution scheme is not new and has already been used by many researchers, as will be shown in the literature review, please see Section 2, the new major feature introduced here, besides the introduction of the richer variant of the PRP, is the adoption of an implicit cost to estimate delivery decisions. This cost acts as a guide to obtain improved solutions by estimating how each delivery will affect the current costs of production, storage, and routing costs. Moreover, we devised different perturbation mechanisms prioritizing either tactical or operational decisions, or both. Finally, we carried out extensive computational experiments to assess the proposed algorithms. The results show that the methods outperformed CPLEX in finding more, better, and faster solutions, having the algorithm with the perturbation procedure that prioritizes operational level decisions outperformed the others.

The remainder of this paper is organized as follows. Section 2 reviews recent works on PRP, while Section 3 provides a formal description of the proposed problem and the adopted notation and definitions. The devised matheuristic approaches are presented in Section 4, whereas Section 5 presents the computational results and extensive statistical analysis. Section 6 concludes with our final remarks.

2. Literature review

According to [Bard and Nananukul \(2010\)](#), PRP has four critical decisions to be made: (a) how many items to manufacture each period, (b) when to visit each customer, (c) how much to deliver to a customer during a visit, and (d) the delivery routes to use. We can also include, as a fifth decision, (e) the amount of each product to be stored at the manufacturer plants and clients. These decisions belong to two different decision levels, a tactical responsible for production, storage,

and back-ordering plans, and an operational one responsible for product distribution and routing. The work of [Adulyasak et al. \(2015b\)](#) extensively studied PRP by listing the main devised solution methods, while the works of [Reimann et al. \(2014\)](#) and [Díaz-Madroño et al. \(2015\)](#) have discussed PRP and its many variants. Here, we discuss the main works that came after these literature reviews.

[Adulyasak et al. \(2014b\)](#) were the first to apply the adaptive large neighborhood search (ALNS) to the classic PRP, achieving results that outperformed the existing heuristics at that time for most tested instances. [Belo-Filho et al. \(2015\)](#) addressed a case with perishable goods. They also adopted ALNS but with a simulated annealing acceptance criterion.

Decomposition approaches are among the most intuitive and widely adopted solution strategies in the PRP literature ([Adulyasak et al., 2015b](#)). Following this concept, [Absi, Archetti, Dauzère-Pérès, and Feillet \(2014\)](#) designed a two-phase iterative mixed-integer program (MIP)-based heuristic to solve the uncapacitated version of the problem. When compared with [Archetti et al. \(2011\)](#) and [Adulyasak et al. \(2014b\)](#), their algorithm outperformed both. However, the [Absi et al. \(2014\)](#) procedure was later outperformed in most sets of solution quality but spending more time by [Solyali and Süral \(2017\)](#), while [Russell \(2017\)](#) achieved equivalent or superior results. Working with a five-phase heuristic, [Solyali and Süral \(2017\)](#) considered a tour idea solved via a TSP. [Russell \(2017\)](#) adopted a matheuristic based on predetermined or seeded routes. They used the concept of a reactive tabu search to improve the previously generated routes with artificial demands.

[Miranda et al. \(2018\)](#) and [Qiu, Wang, Xu et al. \(2018\)](#) split the classical PRP into lot-sizing and vehicle-routing problems, wherein the former was solved with a MIP solver whereas the latter, heuristically. [Miranda et al. \(2018\)](#) presented a formulation with products sharing the production line during some periods. Their method achieved near-optimal solutions when compared with CPLEX. [Qiu, Wang, Xu et al. \(2018\)](#) applied a skewed general variable neighborhood search and guided variable neighborhood descent to the delivery and routing decisions, respectively. Their method outperformed previous existing methods for solving the benchmark instances of [Archetti, Bertazzi, Laporte, and Speranza \(2007\)](#) (standard and high transportation costs) and [Boudia et al. \(2007\)](#) (large size), with comparable computational times.

[Brahimi and Aouam \(2016\)](#) were the first authors to address the PRP considering the possibility of back-orders. They used a relax-and-fix heuristic and used it to build a solution for the lot-sizing problem. Their heuristic outperformed the CPLEX, solving the two formulations proposed while providing good solutions in a few minutes. [Watanabe et al. \(2017\)](#) used the relax-and-fix heuristic to obtain an initial solution and a fix-and-optimize approach to improve solutions of a classical PRP. Promising results were observed for instances with up to 30 customers. [Neves-Moreira, Almada-Lobo, Cordeau, Guimarães, and Jans \(2019\)](#) solved a large multi-perishable-item PRP considering time windows for a real meat producer with a three-phase fix-and-optimize methodology.

[Adulyasak et al. \(2014a\)](#) introduced multi-vehicle formulations with and without vehicle-index for the classical PRP. They solved the formulations using a branch-and-cut (B&C) algorithm. The vehicle-indexed formulation was superior in obtaining optimal solutions, whereas the non-vehicle-index formulation generally provided better lower bounds on larger instances that did not reach optimality. The B&C algorithm attained better lower bounds at the root node of the branch-and-bound search tree, as well as the best overall gaps. [Adulyasak et al. \(2015a\)](#) were the first authors to solve the PRP under demand uncertainty, considering scenarios of two or multiple stages. They devised a [Benders \(1962\)](#) decomposition algorithm embedded in a B&C scheme. The computational results showed that their method was more efficient in handling instances with a greater number of scenarios than CPLEX.

Recently, there has been concern about the impact of supply chain activities on the environment. Carbon footprints and green/reverse logistics have attracted the attention of several researchers. A PRP

approach for carbon and cap trade and partial delivery with lost sales was proposed by Qiu et al. (2017). They assumed an objective function wherein the costs of the generated emissions were proportional to the carbon price. Their model proved capable of simultaneously reducing the emission levels of carbon dioxide and operational costs. Fang et al. (2017) investigated a PRP with a reverse logistic also considering simultaneous pickup and delivery assumption to reduce carbon emissions. Qiu, Ni et al. (2018) studied a PRP with reverse logistics and remanufacturing conditions.

In recent years, more variants of PRP have been proposed that consider aspects to make the problem more appropriate to certain situations. Chitsaz, Cordeau, and Jans (2019) addressed the assembly routing problem that combines the production decisions with inbound logistics activities. The proposed algorithm was a three-phase decomposition matheuristic that outperformed the state-of-the-art heuristics at that time. Considering a multi-product PRP with outsourcing, Li, Chu, Chu, and Zhu (2019) developed a three-level heuristic. The efficiency of the algorithm was proved using tests over large instances and providing new best solutions for 283 of the 1530 benchmark instances.

Avci and Yildiz (2019) addressed the PRP with visit spacing policy to obtain higher quality service solutions. To solve this problem, a matheuristic approach based on a multi-start with an MIP iterative local search was proposed. When solving some standard PRP instances, their algorithm performed better than those of Absi et al. (2014), Adulyasak et al. (2014b), Li et al. (2019), Qiu, Wang, Xu et al. (2018), Solyali and Süral (2017), and Chitsaz et al. (2019).

Dayarian and Desaulniers (2019) proposed a caterer PRP that considered short-life-span product(s) that are produced and consumed within a day. A branch-price-and-cut (BP&C) algorithm was developed to solve small instances of the problem. They also devised three matheuristics capable of solving larger instances; nonetheless, all had similar average gaps.

Avci and Yildiz (2020) addressed the first PRP considering transshipment with single or multiple vehicles. The problem was solved using a fast mathematical programming-based heuristic that obtained high-quality solutions. Golesefidi and Jorak (2020) developed a robust approach to a PRP with uncertainty of the demand, as well as simultaneous pickup and delivery. The problem was solved with simulated annealing and genetic algorithms, performing the latter better than the former. Li, Chu, Côté, Coelho, and Chu (2020) introduced a multi-plant PRP with perishable products with packaging considerations. The problem was solved using a hybrid method and B&C algorithms, with the hybrid method obtaining good solutions within a short computational time. Motivated by a real-life problem in the petrochemical industry, Schenekemberg, Scarpin, Pecora Jr, Guimarães, and Coelho (2021) presented a two-echelon PRP. An exact algorithm wherein local search procedures were incorporated within a B&C framework was proposed to solve this problem.

Table 1 summarizes the information of reviewed papers with regards to the characteristics on production, inventory, distribution, and routing, besides the solution approaches. Production considers whether there are single or multiple plants (products) as well as capacity constraints. The inventory defines whether there are limits to storage as well as whether it works with the maximum level or the order-up-to policies (Coelho et al., 2013). Distribution considers whether the delivered loads can be realized by different vehicles in a given time period t , and if the demand can be back-ordered to a future period. Routing determines whether the fleet is homogeneous or heterogeneous in terms of capacity, and if the number of vehicles is unlimited, limited, single, or multiple (Díaz-Madroño et al., 2015). The solution distinguishes if the adopted approach is exact, heuristic, metaheuristic, or hybrid blending exact and heuristic methods in some manner.

On observing Table 1, it is clear that there is a lack of studies that consider the ideas of back-ordering deliveries and routing decisions for a heterogeneous fleet, features that are commonly observed in practice. In addition, hybrid methods that take advantage of combinations of mathematical procedures and heuristics have been extensively explored. In this manner, our work proposes a richer problem and solution algorithms that contribute to fulfilling these gaps.

3. Problem description and proposed formulation

The richer PRP here addressed relies on the following notations and definitions. Let $\mathcal{T} = \{1, \dots, T\}$ be a set of finite and discrete planning horizon. Let $\mathcal{N} = \{1, \dots, n\}$ be the set of clients supplied by a plant with products of the set $\mathcal{P} = \{1, \dots, P\}$. Each client $i \in \mathcal{N}$ demands $d_{ik}^t \geq 0$ units of product $k \in \mathcal{P}$ in period $t \in \mathcal{T}$. Let $\mathcal{N}' = \{0\} \cup \mathcal{N}$ be the set of nodes with the plant labeled 0 and its copy $n+1$.

Every time period $t \in \mathcal{T}$, the plant manufactures product $k \in \mathcal{P}$, while satisfying production capacity C^k , it incurs a setup l^k and u^k unit costs. A product $k \in \mathcal{P}$ can be stored at the plant or clients up to the limit of U_i^k units, $\forall i \in \mathcal{N}$ but incurs a unitary holding cost h_{ik}^t for each period $t \in \mathcal{T}$. Every time the demand of client $i \in \mathcal{N}$, for product $k \in \mathcal{P}$, cannot be fulfilled in a timely fashion in a period $t \in \mathcal{T}$, the unfulfilled quantity can be back-ordered but at a penalty unitary cost B_{ik}^k .

A limited heterogeneous fleet, given by a set of vehicles $\mathcal{V} = \{1, \dots, V\}$ delivers the products via routes that start at the plant node 0 and end at its copy $n+1$, at the end of each period $t \in \mathcal{T}$. Every vehicle $v \in \mathcal{V}$ that starts a route in a time period $t \in \mathcal{T}$ incurs an activation cost e^v , and the routing cost associated with the arcs of the directed graph $G = (\mathcal{N}_0, \mathcal{A})$, where $\mathcal{N}_0 = \mathcal{N}' \cup \{n+1\}$ and $\mathcal{A} = \{(i, j) : i, j \in \mathcal{N}_0, i \neq j\}$. Every arc $(i, j) \in \mathcal{A}$ has a routing cost $c_{ij} > 0$ and traversing time $a_{ij} > 0$. To unload the products from the vehicles at the clients, it takes a total of s_i units of service time.

To ease the reading, Table 2 lists the parameters of the problem with parameters $\bar{M}_k^t$ and $\bar{M}_{ik}^{vt}$ representing the maximum quantities for manufacturing and delivering the product $k \in \mathcal{P}$ to client $i \in \mathcal{N}$ using vehicle $v \in \mathcal{V}$ in period $t \in \mathcal{T}$, respectively.

A vehicle-indexed formulation is here proposed. This model extends the model presented in Adulyasak et al. (2015b) to our rich production-routing problem (RPRP). It uses the following decision variables. The production-related variables are variable $y_k^t \in \{0, 1\}$ is equal to one if a setup is realized to manufacture $p_k^t \geq 0$ units of product $k \in \mathcal{P}$ in period $t \in \mathcal{T}$, and zero otherwise. The variable $I_{ik}^t \geq 0$ indicates the inventory level for product $k \in \mathcal{P}$ at node $i \in \mathcal{N}$ in period $t \in \mathcal{T}$. The variable $b_{ik}^t \geq 0$ represents the quantity of unfilled demand for product $k \in \mathcal{P}$ at node $i \in \mathcal{N}$ in period $t \in \mathcal{T}$.

The counterpart routing variables are variable $g^{vt} \in \{0, 1\}$ is equal to one if vehicle $v \in \mathcal{V}$ is set to serve the clients' demands of period $t \in \mathcal{T}$; otherwise, it is zero. The variable $z_i^{vt} \in \{0, 1\}$ is equal to one if client $i \in \mathcal{N}$ is visited by vehicle $v \in \mathcal{V}$ to deliver $q_{ik}^{vt} \geq 0$ units of product $k \in \mathcal{P}$ in period $t \in \mathcal{T}$; otherwise, it is zero. Finally, the variable $x_{ij}^{vt} \in \{0, 1\}$ is equal to one if arc $(i, j) \in \mathcal{A}$ is used by vehicle $v \in \mathcal{V}$ in period $t \in \mathcal{T}$; otherwise, it is zero. The variable $w_{ij}^{vt} \geq 0$ represents the arrival time at node $j \in \mathcal{N}_0$ of vehicle $v \in \mathcal{V}$ after traversing arc $(i, j) \in \mathcal{A}$ in period $t \in \mathcal{T}$. To facilitate the representation, Table 3 lists all decision variables. With the aforementioned notation and definitions, the studied RPR can be formulated as

$$\min \sum_{t \in \mathcal{T}} \left\{ \sum_{k \in \mathcal{P}} \left[l^k y_k^t + u^k p_k^t + \sum_{i \in \mathcal{N}} h_{ik}^t I_{ik}^t + \sum_{i \in \mathcal{N}} B_{ik}^k b_{ik}^t \right] + \sum_{v \in \mathcal{V}} \left[e^v g^{vt} + \sum_{(i,j) \in \mathcal{A}} c_{ij} x_{ij}^{vt} \right] \right\} \quad (1)$$

s.t.:

$$p_k^t \leq \bar{M}_k^t y_k^t, \quad \forall k \in \mathcal{P}, t \in \mathcal{T} \quad (2)$$

$$I_{ik}^t = I_{0k}^{t-1} + p_k^t - \sum_{i \in \mathcal{N}} \sum_{v \in \mathcal{V}} q_{ik}^{vt}, \quad \forall k \in \mathcal{P}, t \in \mathcal{T} \quad (3)$$

$$I_{ik}^t = I_{ik}^{t-1} + \sum_{v \in \mathcal{V}} q_{ik}^{vt} - d_{ik}^t + b_{ik}^t - b_{ik}^{t-1}, \quad \forall i \in \mathcal{N}, k \in \mathcal{P}, t \in \mathcal{T} \quad (4)$$

$$I_{ik}^t \leq U_i^k, \quad \forall i \in \mathcal{N}, k \in \mathcal{P}, t \in \mathcal{T} \quad (5)$$

$$q_{ik}^{vt} \leq \bar{M}_{ik}^{vt} z_i^{vt}, \quad \forall i \in \mathcal{N}, k \in \mathcal{P}, v \in \mathcal{V}, t \in \mathcal{T} \quad (6)$$

$$\sum_{k \in \mathcal{P}} \sum_{i \in \mathcal{N}} q_{ik}^{vt} \leq Q^v g^{vt}, \quad \forall v \in \mathcal{V}, t \in \mathcal{T} \quad (7)$$

$$\sum_{v \in \mathcal{V}} z_i^{vt} \leq 1, \quad \forall i \in \mathcal{N}, t \in \mathcal{T} \quad (8)$$

Table 1
Summary of the reviewed papers.

AUTHORS	CHARACTERISTICS											
	PRODUCTION			INVENTORY		DISTRIBUTION		ROUTING			SOLUTION	
	Plants	Product	Capacity	Policy	Capacity	Partial	Back-ordering	Vehicles	Fleet	Capacity	Type	Approach
Absi et al. (2014)	S	S	no	ML	yes	no	no	M	HO	yes	HR	Decomposition
Adulyasak et al. (2014a)	S	S	yes	ML/OU	yes	no	no	M	HO	yes	EX	B&C
Adulyasak et al. (2014b)	S	S	yes	ML	yes	no	no	M	HO	yes	HR	ALNS
Adulyasak et al. (2015a)	S	S	yes	ML/OU	yes	no	no	M	HO	yes	EX	Benders/B&C
Belo-Filho et al. (2015)	M	M	yes	–	yes	no	no	M	HO	yes	HR	ALNS
Brahimi and Aouam (2016)	S	M	yes	ML	yes	no	yes	L	HO	yes	HR	RF
Fang et al. (2017)	S	S	yes	ML	yes	no	no	S	HO	yes	EX	B&C
Qiu et al. (2017)	S	S	yes	ML	yes	yes	no	M	HO	yes	EX	B&P
Russell (2017)	S	S	yes	ML	yes	no	no	M	HO	yes	HY	MIP metaheuristic
Solyali and Süral (2017)	S	S	yes	ML	yes	no	no	M	HO	yes	HR	Decomposition
Watanabe et al. (2017)	S	M	yes	ML	yes	no	no	L	HO	yes	HR	RF+FO
Darvish, Archetti, and Coelho (2018)	S	S	yes	ML	yes	no	no	S	HO	yes	EX	B&C
Miranda et al. (2018)	S	M	yes	ML	yes	no	no	M	HE	yes	HY	Decomposition
Qiu, Ni et al. (2018)	M	S	yes	ML	yes	no	no	M	HO	yes	EX	B&C
Qiu, Wang, Fang et al. (2018)	S	S	yes	ML	yes	no	no	M	HO	yes	EX	B&C
Qiu, Wang, Xu et al. (2018)	S	S	yes	ML	yes	no	no	M	HO	yes	HY	VNS+VND
Avci and Yildiz (2019)	S	S	yes	ML	yes	no	no	S/M	HO	yes	HY	Multi-start and ILS
Chitsaz et al. (2019)	S	S	yes	ML	yes	no	no	M	HO	yes	HY	Three-phase matheuristic
Dayarian and Desaulniers (2019)	S	M	yes	ML	yes	no	no	M	HO	yes	EX/HY	BP&C/Matheuristics
Li et al. (2019)	S	M	yes	ML	yes	no	no	M	HO	yes	HR	Three-level heuristic
Neves-Moreira et al. (2019)	S	M	yes	ML	yes	no	no	M	HE	yes	HR	FO
Avci and Yildiz (2020)	S	S	yes	ML	yes	no	no	S	HO	yes	HY	Mathematical based heuristic
Golsefid and Jokar (2020)	S	S	yes	ML	yes	no	no	U	HO	yes	HR	GA/SA
Li et al. (2020)	M	S	yes	ML	yes	no	no	M	HO	yes	HY/EX	B&C/Matheuristic
Schenekemberg et al. (2021)	M	M	yes	ML/OU	yes	no	no	M	HO	yes	EX/HY	B&C/LS
<i>This study</i>	S	M	yes	ML	yes	no	yes	L	HE	yes	HY	ILS matheuristic

(S) single, (M) multiple, (ML) maximum level, (OU) order-up-to, (HO) homogeneous, (HE) heterogeneous, (L) limited, (U) unlimited, (EX) exact, (HR) heuristic or metaheuristic, (HY) hybrid.

Table 2
Problem parameters.

Notation	Meaning
C^k, l^k, u^k	Production capacity, setup, and unitary costs of product $k \in \mathcal{P}$
U^k, h_i^k, B_i^k	Inventory capacity, holding and back-order costs of product $k \in \mathcal{P}$ for client $i \in \bar{\mathcal{N}}$
e^v, Q^v	Cost and capacity of each vehicle $v \in \mathcal{V}$
c_{ij}, a_{ij}	Connection costs and traveling times of each arc $(i, j) \in \mathcal{A}$
H, s_i	Maximum riding time and service times of each customer $i \in \bar{\mathcal{N}}$
d_{ik}^t	Periodic demand of each client $i \in \bar{\mathcal{N}}$ of product $k \in \mathcal{P}$ in period $t \in \mathcal{T}$
$\bar{M}_k = \min\{C_k, \sum_{i \in \bar{\mathcal{N}}} \sum_{t \in \mathcal{T}} d_{ik}^t\}, \forall i \in \bar{\mathcal{N}}, k \in \mathcal{P}$	
$\bar{M}_{ik}^{vt} = \min\{U_i^k, Q^v, \sum_{t \in \mathcal{T}} d_{ik}^t\}, \forall i \in \bar{\mathcal{N}}, k \in \mathcal{P}, v \in \mathcal{V}, t \in \mathcal{T}$	

Table 3
Decision variables of the vehicle-indexed formulation.

Notation	Meaning
$b_{ik}^{vt} \geq 0$	Quantity back-ordered at customer $i \in \bar{\mathcal{N}}$ of product $k \in \mathcal{P}$ in period $t \in \mathcal{T}$
$I_{ik}^{vt} \geq 0$	Inventory level at node $i \in \mathcal{N}$ of product $k \in \mathcal{P}$ at the end of period $t \in \mathcal{T}$
$p_k^{vt} \geq 0$	Quantity of product $k \in \mathcal{P}$ manufactured in period $t \in \mathcal{T}$
$q_{ik}^{vt} \geq 0$	Quantity of product $k \in \mathcal{P}$ delivered to customer $i \in \bar{\mathcal{N}}$ in period $t \in \mathcal{T}$ using vehicle $v \in \mathcal{V}$
$w_{ij}^{vt} \geq 0$	The arrival time at node $j \in \mathcal{N}_0$ after vehicle $v \in \mathcal{V}$ traverses arc $(i, j) \in \mathcal{A}$ in period $t \in \mathcal{T}$
$g^{vt} \in \{0, 1\}$	It is equal to one if vehicle $v \in \mathcal{V}$ performs a route in period $t \in \mathcal{T}$; otherwise, it is zero
$x_{ij}^{vt} \in \{0, 1\}$	It is equal to one if arc $(i, j) \in \mathcal{A}$ is used by vehicle $v \in \mathcal{V}$ in period $t \in \mathcal{T}$; otherwise, it is zero
$y_k^{vt} \in \{0, 1\}$	It is equal to one if the product $k \in \mathcal{P}$ is manufactured in period $t \in \mathcal{T}$; otherwise, it is zero
$z_i^{vt} \in \{0, 1\}$	It is equal to one if node $i \in \mathcal{N}$ is visited by vehicle $v \in \mathcal{V}$ in period $t \in \mathcal{T}$; otherwise, it is zero

$$z_0^{vt} + z_{n+1}^{vt} = 2g^{vt}, \quad \forall v \in \mathcal{V}, t \in \mathcal{T} \quad (9)$$

$$\sum_{i \in \bar{\mathcal{N}}} x_{0i}^{vt} = z_0^{vt}, \quad \forall v \in \mathcal{V}, t \in \mathcal{T} \quad (10)$$

$$\sum_{(j,i) \in \mathcal{A}} x_{ji}^{vt} = z_i^{vt}, \quad \forall i \in \bar{\mathcal{N}}, v \in \mathcal{V}, t \in \mathcal{T} \quad (13)$$

$$w_{0i}^{vt} = a_{0i} x_{0i}^{vt}, \quad \forall i \in \bar{\mathcal{N}}, v \in \mathcal{V}, t \in \mathcal{T} \quad (14)$$

$$\sum_{i \in \bar{\mathcal{N}}} x_{i,n+1}^{vt} = z_{n+1}^{vt}, \quad \forall v \in \mathcal{V}, t \in \mathcal{T} \quad (11)$$

$$\sum_{(i,j) \in \mathcal{A}} w_{ij}^{vt} - \sum_{(j,i) \in \mathcal{A}} w_{ji}^{vt} = \sum_{(i,j) \in \mathcal{A}} (s_i + a_{ij}) x_{ij}^{vt}, \quad \forall i \in \bar{\mathcal{N}}, v \in \mathcal{V}, t \in \mathcal{T} \quad (15)$$

$$\sum_{(i,j) \in \mathcal{A}} x_{ij}^{vt} = z_i^{vt}, \quad \forall i \in \bar{\mathcal{N}}, v \in \mathcal{V}, t \in \mathcal{T} \quad (12)$$

$$0 \leq w_{ij}^{vt} \leq H x_{ij}^{vt}, \quad \forall (i, j) \in \mathcal{A}, v \in \mathcal{V}, t \in \mathcal{T} \quad (16)$$

$$p_k^t, I_{ik}^t, b_{ik}^t, q_{ik}^{vt} \geq 0, y_k^t \in \{0, 1\}, \quad \forall i \in \mathcal{N}, k \in \mathcal{P}, v \in \mathcal{V}, t \in \mathcal{T} \quad (17)$$

$$g^{vt}, z_i^{vt} \in \{0, 1\} \quad \forall i \in \mathcal{N}, v \in \mathcal{V}, t \in \mathcal{T} \quad (18)$$

$$x_{ij}^{vt} \in \{0, 1\}, w_{ij}^{vt} \geq 0, \quad \forall (i, j) \in \mathcal{A}, v \in \mathcal{V}, t \in \mathcal{T} \quad (19)$$

The objective function (1) minimizes the production, setup, holding, back-ordering, fleet, and routing costs. Constraints (2) limit the quantity to be produced for each item either by the plant's production capacity for that item or by its total remaining demand for horizon planning. Constraints (3)–(5) control the inventory level of each product in each period. For each product, $k \in \mathcal{P}$, both constraints (3) and (4) guarantee the balance of the inventory level at the end of period $t \in \mathcal{T}$ at the plant and customers, respectively. The maximum inventory levels at the factory and clients are stated via (5). The distribution and routing constraints are given by (6)–(16). The quantity of each item $k \in \mathcal{P}$ delivered to each client $i \in \mathcal{N}$ in each period $t \in \mathcal{T}$ is limited by constraints (6). Constraints (7) assure that the vehicles' capacity are not exceeded.

Constraint (8) limits the maximum number of times that a client $i \in \mathcal{N}$ is visited per period $t \in \mathcal{T}$, whereas constraint (9) stipulates that if a vehicle $v \in \mathcal{V}$ leaves the plant node 0, it must return to its node copy $n + 1$. Constraints (10)–(13) are the degree-related constraints for the plant, its node copy, and the clients visited by the same vehicle $v \in \mathcal{V}$ in a period $t \in \mathcal{T}$.

Proposed by Bianchessi et al. (2018), constraints (14)–(16) to bind the riding time of the vehicle while eliminating the subtours. The vehicle arrival times at the clients after leaving the plant are properly determined by constraints (14), while constraints (15) balance the vehicles' time continuity. Constraints (16) bound the vehicles' maximum riding time. Constraints (17)–(19) are related to the variables' domain.

We assume that the back-orders in the last period are equal to zero, as in Brahimi and Aouam (2016). Herein, formulation (1)–(19) is named as VINDX for sake of simplicity.

4. Efficient matheuristics approaches

As the studied RPRP inherits the computational complexity of the embedded capacity routing problem within it, that is, the studied RPRP is NP-hard, we herein present the three devised hybrid algorithms to solve it. Hybrid methods seek to blend the speed of heuristics with the determinism of the exact methods. Such a solution framework is normally called a matheuristic. Matheuristic approaches explore mathematical programming techniques in metaheuristic frameworks or on granting to mathematical programming approaches the cross-problem robustness and constrained-CPU-time effectiveness, which characterize metaheuristics (Caserta & Voß, 2009). The devised algorithms work in two-level decision-making within an iterated local search (ILS) framework (Lourenço, Martin, & Stützle, 2003). The tactical level is responsible for the production and inventory plans, while the operational level plans the distribution of the products and routing of the vehicles. The three proposed hybrid methods follow a top-down hierarchical approach.

4.1. Building a solution

Algorithm 1 provides an initial solution based on the problem information and parameter Δ to the top-down methods. It constructs a feasible plan for the tactical problem and then proposes delivery routes for each period. Throughout the execution of the algorithm, the parameter $\Delta = [\Delta_{ik}^{vt}], \forall i \in \mathcal{N}, k \in \mathcal{P}, v \in \mathcal{V}, t \in \mathcal{T}$, is a vector that has the delivery estimated costs, which plays an important role in our methods. Δ_{ik}^{vt} estimates the impact that one unit of the delivery lot q_{ik}^{vt} will have on the production, inventory, and transportation decisions.

To obtain an initial solution to the problem, Algorithm 1 computes Δ_{ik}^{vt} , as shown in Eq. (20). The visitation cost Δ_i is calculated as in Qiu, Wang, Xu et al. (2018). The other components consider the cost of transporting a unit of product $k \in \mathcal{P}$ by each vehicle $v \in \mathcal{V}$ using the fraction $\frac{e^v}{Q^v}$, and, if produced, whether this unit will be stored either at the plant or at the customer. The parameter $\omega \in [1, \text{maxWeight}]$, belonging to a discrete and uniform distribution, adds a random component to the problem to help escape non-promising search areas.

$$\Delta_{ik}^{vt} = \omega \left(\Delta_i + \frac{e^v}{Q^v} + u^k + \min\{h_0^k, h_i^k\} \right), \forall i \in \mathcal{N}, k \in \mathcal{P}, v \in \mathcal{V}, t \in \mathcal{T} \quad (20)$$

Algorithm 1: Build and optimization

Data: Problem information, costs Δ

- 1 $\bar{q} \leftarrow \text{BuildAndSolve}(\text{PI}_v(\Delta));$
- 2 $\text{VR}_t \leftarrow \emptyset;$
- 3 **for** $t \in \mathcal{T}$ **do**
- 4 $\text{vr}(\bar{q}^t) \leftarrow \text{BuildVRP}(\bar{q}^t);$
- 5 $\text{VR}_t \leftarrow \text{VR}_t \cup \{\text{NRS}(\text{vr}(\bar{q}^t))\};$
- 6 **end**
- 7 $s \leftarrow \text{PI}_v \cup \text{VR}_t;$
- 8 **return** s

Algorithm 1 solves in line 1 the top-tier problem PI_v with the indirect costs Δ_{ik}^{vt} , as expressed in (21)–(23) via CPLEX. The variables and constraints of formulation (21)–(23) have the same meaning as before; please see Table 3. With the adoption of Δ , solving PI_v without the visitation (z) and vehicle activation (g) binary variables is possible because Δ carries both routing information.

$$\min \sum_{t \in \mathcal{T}} \left\{ \sum_{k \in \mathcal{P}} \left[I^k y_k^t + u^k p_k^t + \sum_{i \in \mathcal{N}} h_i^k I_{ik}^t + \sum_{i \in \mathcal{N}} \left(B_i^k b_{ik}^t + \sum_{v \in \mathcal{V}} \Delta_{ik}^{vt} q_{ik}^{vt} \right) \right] \right\} \quad (21)$$

s.t.: (2)–(5)

$$\sum_{k \in \mathcal{P}} \sum_{i \in \mathcal{N}} q_{ik}^{vt} \leq Q^v, \forall v \in \mathcal{V}, \forall t \in \mathcal{T} \quad (22)$$

$$0 \leq q_{ik}^{vt} \leq \bar{M}_{ik}^{vt}, \forall i \in \mathcal{N}, k \in \mathcal{P}, v \in \mathcal{V}, t \in \mathcal{T} \quad (23)$$

After solving PI_v , the attained loads are summed into parameter $\bar{q}_i^t = \sum_{k \in \mathcal{P}} \sum_{v \in \mathcal{V}} q_{ik}^{vt}$, representing the total amount of mixed product to be delivered to client $i \in \mathcal{N}$ in period $t \in \mathcal{T}$. Next, the VR_t problem is initialized (Alg. 1, line 2). This corresponds to the set of independent routing solutions for periods $t \in \mathcal{T}$.

For each period, the BuildVRP function generates feasible routes $\text{vr}(\bar{q}^t)$ (Alg. 1, line 4). This is managed via randomly selected constructive methods chosen within the following: the Clarke–Wright savings heuristic (parallel or sequential, see Clarke & Wright, 1964 and Lysgaard, 1997), sequential-lexicographic, and load-ordered insertions. For all these methods, the vehicles with the largest capacities are filled first. The reason for using these four constructive methods is to allow diversification in the search space. In line 5, the routing solution $\text{vr}(\bar{q}^t)$ is improved by the procedure neighborhood routing search (NRS) procedure, which is described later in Section 4.2, with the resulting routing solution added to the VR_t list of routes for each period. Although the first two constructive routing methods are well known in the literature, the last two are proposed herein.

The sequential-lexicographic insertion starts from a node defined as 1 and attempts to add it to a vehicle v , considering the feasibility of vehicle capacity and riding time. The procedure investigates all customers until no further addition to v is possible. When a vehicle is fully loaded or reaches the maximum riding time, a new vehicle is activated. The procedure stops after the allocation of all clients. The largest vehicles are activated first. The load-ordered insertion algorithm takes the individual lots q_i of each node i and orders them from the largest to the smallest load. The loads are then iteratively added to the largest vehicle available until no further additions can be performed.

A new vehicle is activated, and the process repeated until all loads are loaded.

For instance, suppose that there are five retailers $\bar{\mathcal{N}} = \{i_1, \dots, i_5\}$ with delivery lots equal to $q_1 = 3, q_2 = 2, q_3 = 4, q_4 = 2, q_5 = 1$, respectively, and that there are two vehicles $\mathcal{V} = \{v_1, v_2\}$ with capacities equal to $Q^1 = 7, Q^2 = 5$. The sequential-lexicographic solution is the vehicle v_1 visiting retailers i_1, i_2 and i_4 , while vehicle v_2 visits i_3 and i_5 . For the load-ordered insertion procedure, the attained solution is a vehicle v_1 visiting retailers i_3 and i_1 , while vehicle v_2 visits i_2, i_4 and i_5 .

4.2. Neighborhood routing search (NRS)

The NRS procedure shown in Algorithm 2 applies a set of local searches to a given routing solution. It is based on the random variable neighborhood descent procedure (Penna, Subramanian, & Ochi, 2013; Souza, Coelho, Ribas, Santos, & Merschmann, 2010) that prevents method favoritism and applies each routing local search as a neighborhood structure. The adopted routing local searches are divided into two groups: inter- and intra-routes (Kytöjoki, Nuortio, Bräysy, & Gendreau, 2007).

First, we set RS for inter-route searches (line 1). While this set RS is not empty, that is, while solution improvements are obtained (lines 2–10), an inter-route local search rs is randomly chosen from the set RS and applied to the current solution $vr(\bar{q})$ (line 4). In the case of improvement, an intra-route search procedure was performed (line 6). Otherwise, the selected local search rs is removed from the set RS (line 8).

Algorithm 2: Neighborhood routing search (NRS)

Data: vehicle-routing solution $vr(\bar{q})$

```

1 RS ← inter-route local search list;
2 while RS ≠ ∅ do
3   rs ← random(RS);
4    $vr'(\bar{q}) \leftarrow \text{InterRoute}(rs, vr(\bar{q}))$ ;
5   if  $f(vr'(\bar{q})) < f(vr(\bar{q}))$  then
6      $vr(\bar{q}) \leftarrow \text{IntraRoute}(vr'(\bar{q}))$ ;
7   else
8     RS ← RS \ {rs}
9   end
10 end
11 return  $vr(\bar{q})$ ;
```

Seven neighborhood moves of inter-route local searches were adopted. They are Penna et al. (2013) as follows: Shift(1,0), Swap(1,1), Shift(2,0), Swap(2,1), Swap(2,2), Cross, and K-Shift ($K = 3, 4, 5$).

These moves are realized only if a new route arrangement is feasible and improves the routing cost. In the shift (1,0) move, customer i is transferred from route r_1 to route r_2 . The move Swap(1,1) picks a client i from route r_1 and client j from route r_2 and exchanges them. In Shift(2,0) move, two adjacent customers $i - j$ from route r_1 are transferred to route r_2 . The move Swap(2,1) exchanges two adjacent customers $i - j$ from route r_1 by customer k from route r_2 . The move Swap(2,2) exchanges a pair $i - j$ of adjacent customers belonging to route r_1 by a second pair of adjacent customers $k - l$ from route r_2 .

The cross move removes arcs connecting adjacent $i - j$ clients belonging to a route r_1 and the adjacent clients $k - l$ belonging to route r_2 . Then, the arcs connecting $i - l$ and $k - j$ are inserted. The move K-Shift removes a subset of consecutive K customers from route r_1 and is transferred to the end of route r_2 . It should be pointed out that this move is also applied if the second route is empty.

Intra-route local searches work reordering the visited nodes by each route. We adopted five of them: 1, 2, or 3-point move, 2-opt, and Or-opt ($Or = 3, 4, 5$). Groër et al. (2010) described them as follows: the 1-point move relocates an existing node into a new position. The 2-point move swaps the position of two nodes. The 3-point move swaps the position

of a pair of adjacent nodes with the position of a third node. The 2-opt move removes two edges from the solution and replaces them with two new edges. The Or-opt move removes a string of nodes and inserts it into a new position.

To achieve better results, we also applied the very large-scale neighborhood search (VLNS¹) proposed by Ahuja et al. (2000). It replaces both moves Shift(1,0) and Swap(1,1) with substantially more neighbors. This occurs because VLNS simultaneously contemplates swap and shift moves not only between a pair of routes, as done by the regular shift (1,0) and swap (1,1) moves, but among all sets of routes if the move is feasible. Its design is extremely efficient for the family of set partitioning problems. When the VLNS neighborhood structure can be classified as a network-flow-based improvement algorithm built on an improvement graph that contains all feasible moves of retailers between routes.

The improvement graph for a neighborhood with multiple exchanges is defined over a feasible routing solution s for the problem, which is represented by $G(s)$. Let $r[i_j]$ be a route that contains the client i_j . The graph $G(s)$ is a directed graph with $n + V + 1$ nodes, where n is the number of clients belonging to the V routes and one additional artificial node. In graph $G(s)$, each node $i_1, \dots, i_n$ corresponds to the customers, while nodes $i_{n+1}, \dots, i_{n+m}$ correspond to vehicles/routes. The artificial node is given by i_{n+m+1} . A directed arc $(l, k) \in G(s)$ indicates that client i_l leaves its current route and is transferred to the route that contains customer i_k , that is, to route $r[i_k]$. Simultaneously, the client i_k leaves $r[i_k]$. To build $G(s)$, all pairs of elements $i_l, i_k \in s$ are considered. An arc (l, k) is added to $G(s)$ if and only if (i) the clients i_l and i_k belong to different routes and (ii) the route $r[i_k] \setminus \{i_k\} \cup \{i_l\}$ is feasible. The cost $\bar{c}_{lk}$ of the arc (l, k) is defined by the difference in the cost of the modified route and its previous version, that is, $\bar{c}(r[i_k] \setminus \{i_k\} \cup \{i_l\}) - \bar{c}(r[i_k])$.

Let D be a directed cycle on the improvement graph $G(s)$ if the elements that compose D belong to different partitions. A valid cycle is defined as a directed cycle with a negative cost of $G(s)$. Thus, a valid cycle corresponds to a cyclic or path exchange, leading to an improvement in the objective function of the problem concerning all constraints. If no improvement is found, the search stops. To achieve these improvements, efficiently identifying these valid cycles in $G(s)$ is necessary. Here, it is performed with a label-correcting algorithm that obtains the minimum path from a given (artificial) source node to all nodes of the network. It is possible that valid cycles in $G(s)$ may not be found; nonetheless, a valid path exchange can be found.

Cyclic and path exchanges are a generalization of the two-exchange neighborhood (Thompson & Orlin, 1989). Given a subset of routes, the cyclic exchange removes one node from each route that composes this subset and exchanges them. Fig. 1 illustrates the cyclic and path exchanges. Shifting the nodes i_A, i_D, i_G , and i_J , among their respective partitions P_1, P_2, P_4 , and P_3 in the sequence $i_A \rightarrow i_D \rightarrow i_J \rightarrow i_G \rightarrow i_A$, characterizes a cycle exchange (Fig. 1(a)). While moving the nodes i_A, i_D , and i_G in the sequence $i_A \rightarrow i_D \rightarrow i_G$, among partitions P_1, P_2 , and P_3 , characterizes a path exchange (Fig. 1(b)). The cardinality of the routes involved in the exchange is maintained for the cyclic, but for the path, one route has its cardinality increased by one while the others have it decreased. Further details regarding the VLNS method can be obtained in Ahuja (2017), Ahuja et al. (1998, 2000).

4.3. Perturbation operators

Local searches can become trapped in basins of attraction, leading to non-promising neighborhoods. To avoid this, a solution perturbation is applied, and perturbation operators must be strong enough to allow the exploration of new regions and to prevent their efforts to be undone by

¹ The VLNS algorithm must not be confused with the large neighborhood search (LNS) proposed by Shaw (1998). LNS belongs to the VLNS class of heuristics (Pisinger & Ropke, 2010).

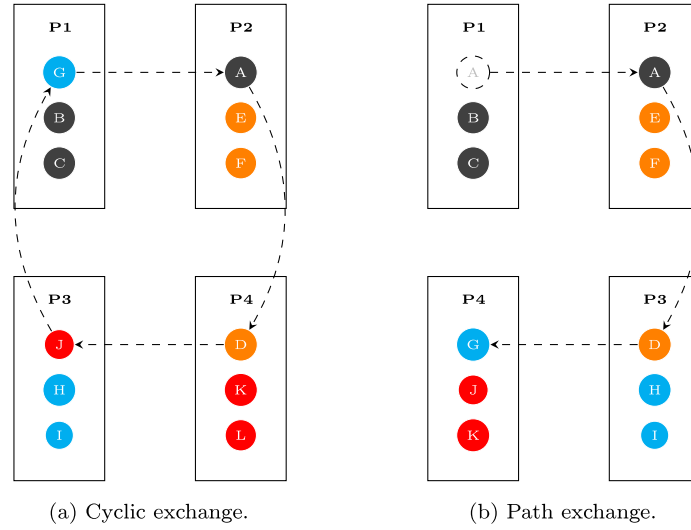


Fig. 1. Very large-scale neighborhood exchanges.

a local search (Lourenço, Martin, & Stützle, 2019). This study considers these recommendations and outlines three algorithms having different perturbation strategies. The main idea of these mechanisms is to modify either or both decision levels while verifying how these changes lead to better solutions.

4.3.1. Production plan operator

It is possible to find in the literature diversification moves, such as the ones proposed by Armentano et al. (2011) and Qiu, Wang, Xu et al. (2018), that only change the distribution plan. Nonetheless, to the best of our knowledge, our production plan (PP) perturbation move is the first one that simultaneously changes the production, holding, and distribution plans. The PP operator shifts part of the lot produced at period t' to a period t'' in which the production also takes place. We consider the periods wherein the production occurs but observe the following conditions: (i) period t'' corresponds to the beginning of the adjacent production stages, (ii) there is idle production capacity at t'' , and (iii) storage capacity available at the plant to store the excess production.

To understand the concept of production stages, let us see the following example. Suppose that there are 14 periods and that production occurs at periods $t = 1, 5, 10$, and 13 . This implies that the production lot manufactured in $t = 1$ fulfills the demands of subsequent periods 1, 2, 3 and 4. The production of period $t = 5$ fulfills the demands of periods 5, 6, 7, 8 and 9. The same reasoning is applied to periods of 10 and 13.

Let us consider that part of the demand $d^{t''}$ of period $t'' = 4$ is back-ordered in $b^{t''}$ units to period $t' = 5$. Then, $d^{t''}$ units will be made in $t' = 5$. Let us also consider that the manufactured lot in period $t' = 10$ contemplates part of the demand of period $t' = 13$. For the sake of the example, let us assume that $I^{12} > d^{12}$. Thus, part of the production lot of period $t' = 10$ can be transferred to $t' = 13$ to meet demand in due time, that is, $I^{12} = 0, p^{13} = d^{13} + d^{14}$.

With this example, it is possible to see that two possible moves can be realized on the production lots, shifting the lot partially or completely to other adjacent periods where production takes place. If $t'' < t'$, then the move anticipates production; otherwise, the production is delayed. Eq. (24) calculates the possible shifted amount, which defines the feasibility of the move. For each product $k \in \mathcal{P}$, it is possible to shift, from period t' to an adjacent period t'' , the minimum value among its entire production lot $p_k^{t'}$, either the residual production capacity $\bar{M}_k^{t''} - p_k^{t''}$ of t'' or the residual holding capacity $U_0^k - I_{0k}^{t''}$ at the plant during t'' .

$$m_k^{t't''} = \min\{p_k^{t'}, \bar{M}_k^{t''} - p_k^{t''}, U_0^k - I_{0k}^{t''}\}, \forall k \in \mathcal{P}, t', t'' \in \mathcal{T}, t'' \neq t' \quad (24)$$

Every time the operator PP is selected, it lists all feasible moves of the production lots. Subsequently, it randomly draws up to $\max Pert PP$ moves for different products and periods. To maintain solution feasibility, it allows the modification of the production plan of a product $k \in \mathcal{P}$ up to one time per perturbation call. The selected moves are applied to PI_v by changing the bounds of the variables while considering the value of Δ_{ik}^{vt} .

4.3.2. Routing operators

Exchanges on the routing plans characterize the perturbation on the operational level. Five operators are responsible to modify the routes. For each of them, between 25% and 75% of the periods are randomly selected and perturbed, but the solutions are always feasible. They are separated into intra-route and inter-route operators. After the execution of routing perturbations, the NRS (Section 4.2) procedure was applied to the perturbed solutions.

The intra-routes perturbations modify one route per time. We devised the reverse-route (RR), lexicographic (LX), and randomized route (RD). Operator RR inverts the sense of selected routes, giving more opportunity to the last visited nodes to be explored first by routing local searches. It reverses the sense of up to $\max V$ routes for a period of time $t \in \mathcal{T}$. This parameter is randomly selected from the interval $[0, V]$ every time the operator is called. As an example, suppose that route r visits a subset of customers in the following order 3, 4, 5, 1, 2. According to the selected intra-route operator, r would turn into 2, 1, 5, 4, 3 applying RR, and operator LX would reorder r in the sequence 1, 2, 3, 4, 5, while RD could generate a new r in the order 3, 1, 4, 2, 5. Both operators LX and RD were calibrated. They are executed in each selected period $t \in \mathcal{T}$ up to $\max Pert LX$ and $\max Pert RD$ times, respectively.

The inter-route perturbation operators modify two routes. The split-route (SR) operator looks for an empty vehicle and allocates to some or all retailers visited by another route. For example, a route that visits the following customers 1, 2, 4, 5, 3 could be divided into 1, 2, 4 and 5, 3. This operator searches for an empty vehicle $v_1 \in \mathcal{V}$ and randomly selects another vehicle $v_2 \in \mathcal{V}, v_1 \neq v_2$, which is active. It then transfers as much as possible to the load from v_1 to v_2 .

The KK operator randomly selects two routes and strings of sequential nodes with sizes K_1 and K_2 belonging to them. These strings are exchanged between routes, allocating them at the end of each route. For example, suppose that a route visiting clients 1, 2, 3, 4 and a second route visiting 7, 6, 8, 5. By applying operator KK, they could be turned into 1, 2, 8, 5 and 7, 6, 3, 4. This perturbation operator is applied up to $\max Pert KK$ over each selected time period $t \in \mathcal{T}$.

4.4. Updating Δ

Every time a perturbation operator is applied to the solution, the indirect costs Δ are updated according to Eq. (25). They offer much more accurate information about the problem and assess the impact that the transported lots have on the production, storage, and routing components.

$$\Delta_{ik}^{vt} = \omega \left(\frac{\Delta_i}{\Delta_r} + \frac{e^v}{Q^v} + \frac{h_i^k}{(I_{ik}^t - I_{ik}^{t-1})} + \frac{u^k}{(\bar{p}_k^t - \bar{p}_k^{t-1})} \right), \forall i \in \bar{\mathcal{N}}, k \in \mathcal{P}, v \in \mathcal{V}, t \in \mathcal{T} \quad (25)$$

Eq. (25) is the sum of the following components. The parameter $\Delta_i = c_{hi} + c_{ij}$, $\forall i, j, k \in r \subset \mathcal{N}$; that is, it is equal to the sum of the costs of the corresponding arcs that connect node i within route r , whose total length is equal to Δ_r . The second component is the fraction $\frac{e^v}{Q^v}$. This shows the cost of transporting a unit by each vehicle $v \in \mathcal{V}$. The component that divides the holding cost h_i^k by the difference between the inventory levels in periods t and $t-1$ estimates the impact of each load unity on the inventory from one period to the next. The reasoning is analogous for the last component, considering production costs and manufactured lots.

By adding more information to Δ , the decision-making at the tactical level is enriched and is able to achieve better solutions when compared with the one proposed by Qiu, Wang, Xu et al. (2018) (see Section 5.4) of the computational experiments.

4.5. Iterated local search framework

Our solution strategy for the studied RPRP is based on an ILS meta-heuristic, whose core concept involves improving a current solution by generating new initial solutions through perturbations combined with local searches (Lourenço et al., 2019).

The devised hybrid approaches follow a top-down decision-making process usually adopted by production planning systems (Bitran & Tirupati, 1993). Algorithm 3 shows the main steps of the devised ILS framework. An initial solution s^* is provided by Algorithm 1 and is declared as the current best solution (line 1). Each iteration of the outer loop (lines 2–20) attempts to improve s^* . The setup variables y have fixed values for each period to the corresponding products being manufactured in solution s^* (line 4). It aims to make the resulting problem easier and quicker to be solved. Line 5 initializes the set P of perturbation operators e^k .

In the inner loop (lines 6–19), a perturbation operator is randomly selected to avoid favoritism and applied to the current solution. The chosen perturbation operator modifies either the tactical or operational part of the solution, or both. Recall the operators were presented in Section 4.3.

The parameter Δ and its updating function are at the core of the proposed ILS algorithms. As previously mentioned, parameter Δ is a vector of indirect costs, which is used for the to-be-delivered loads, and to guide the solution of the tactical problem PI_v . As the RPRP is solved in two stages, parameter Δ estimates the impact of the delivery lots on the production and routing plans. It works as a feedback parameter to guide the solution of the top-tier problem. Function update Δ renews parameter Δ at the problem PI_v (see Section 4.4).

In line 10, the problem PI_v is optimized considering the renewed Δ and examined if it found a better objective function value. If a better solution is obtained, the search continues from it. Otherwise, the objective function value of the current solution is tested, and if it is less than $(1 + \alpha)$ times the objective function value, this solution is explored in the next iteration. This step allows for diversification, exploring non-local optimal solutions, and is inspired by the skewed variable neighborhood search (Hansen & Mladenović, 2001).

A perturbation operator is discarded from the list if a solution improvement is not attained in the current iteration. This strategy of

the perturbation operator can be understood as a combination of the cyclic and pipe neighborhood exchange steps (Hansen, Mladenović, Todosijević, & Hanafi, 2017). If there is an improvement, the algorithm does not immediately return to the first perturbation operator, that is, it can explore a different perturbation operator from the current operator unless it is drawn again. However, if there is no improvement, the remaining operators have the same opportunity to be selected. The results show that no operator was at a disadvantage to the others.

If the setup variables y at the tactical problem are fixed, then they are unfixed. Otherwise, they were fixed. It works closely related to a relax-and-fix procedure (Pochet & Wolsey, 2006). Recall that, with y fixed, it is expected that the problem will be solved easily and quickly.

Algorithm 3: Iterated local search (ILS)

Data: Problem information, costs Δ

```

1  $s^* \leftarrow \text{BuildAndOptimize}(\Delta)$ ;
2 for  $i \leftarrow \text{maxIter}$  do
3    $s \leftarrow s^*$ ;
4   fix the setup variables  $y$ ;
5    $P \leftarrow \{e^k\}$ ;
6   while  $P \neq \emptyset$  do
7      $e^k \leftarrow \text{Random}(P)$ ;
8      $s' \leftarrow \text{Perturb}(s, e^k)$ ;
9      $s' \leftarrow \text{Update}\Delta(s')$ ;
10     $s'' \leftarrow \text{Opt}(s')$ ;
11    if  $f(s'') < f(s)$  then
12       $s \leftarrow s''$ ;
13      if  $f(s) < f(s^*)$  then  $s^* \leftarrow s$ ;
14    else
15       $P \leftarrow P \setminus \{e^k\}$ ;
16      if  $f(s'') < (1 + \alpha)f(s)$  then  $s \leftarrow s''$ ;
17      unfix/fix  $y$ ;
18    end
19  end
20 end
21 return  $s^*$ ;

```

The proposed algorithms were named TILS, OILS, and IILS, according to the adopted perturbation strategy: tactical, operational, and integrated, respectively. TILS adopted the production plan operator PP from Section 4.3.1. OILS selects within the five routing operators – RR, LX, RD, SR, and KK – as presented in Section 4.3.2. Finally, IILS uses all proposed perturbation operators.

5. Computational results and analysis

This section reports the computational experiments performed with all analyses performed for our algorithms. Here, we present a summary of our results, with all data available in the online resource provided by the authors.

5.1. Instance generation

To the best of our knowledge, there is no set of benchmark instances available that gather all the necessary information for our formulation and algorithms. Therefore, we have generated a set of instances that were created using an approach similar to the works of Coelho and Laporte (2013) and Brahimi and Aouam (2016), but our instances are not the same or equivalent to theirs, nonetheless, since we have addressed a much richer problem here. We have developed 108 instances by varying four main aspects: (1) number of retailers ($n = 10c, c \in \{2, \dots, 5\}$), (2) number of periods ($T \in \{5, 10, 15\}$), (3) number of products ($P \in \{3, 5, 7\}$), and (4) number of available vehicles ($V \in \{7, 9, 11\}$).

The parameters and costs were drawn from discrete uniform distributions within the intervals, as shown in Table 4. Two different intervals were used to characterize low and high levels of demands. We

Table 4
Parameters and costs values.

Values	Meaning
$x_i, y_i \in [0, 1000], \forall i \in \mathcal{N}$	The node coordinates
$d_{ik}^t \in [0, 25]$ or $d_{ik}^t \in [30, 55], \forall (i, k, t) \in \{\mathcal{N}, P, T\}$	Clients' periodic demand
$I_{0k}^0 \in [100, 150], \forall k \in \mathcal{P}$	Initial inventory of the plant
$C_k = nA_k, A_k \in [50, 140], \forall k \in \mathcal{P}$	Production capacity
$u^k \in [2, 8], t^k = 10^4 u^k, \forall k \in \mathcal{P}$	Production setup and unitary costs
$U_i^k = \kappa F_k : F_k \in [140, 190], \kappa \in \{2, 3, 4\}, \forall (i, k) \in \{\mathcal{N}, P\}$	Holding capacity
$h_0^k \in [1, 5], h_i^k \in [6, 10], \forall (i, k) \in \{\mathcal{N}, P\}$	Holding cost
$B_i^k = \gamma h_i^k, \gamma \in \{8, \dots, 12\}, \forall (i, k) \in \{\mathcal{N}, P\}$	Backorder costs
$\bar{D} = \left\lfloor \frac{\sum_{i \in \mathcal{N}} \sum_{k \in \mathcal{P}} \sum_{t \in T} d_{ik}^t}{ T } \right\rfloor$	Average demand per period,
$e^v \in [500, 1000], Q^v \in \frac{2\bar{D}}{T} [0.8, 1], \forall v \in \mathcal{V}$	Vehicle cost and capacity
$c_{ij} = \text{round}(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}), \forall (i, j) \in \mathcal{A}$	Traveling costs
$a_{ij} = c_{ij}, \forall (i, j) \in \mathcal{A}$	The traveling times
$H = 6000$	The maximum riding time
$s_i = 50, \forall i \in \mathcal{N}$	client service time,

assumed, as in [Brahimi and Aouam \(2016\)](#), that the initial back-order was equal to zero, and the inventory at the clients was equal to the clients' first-period demand.

At the link <https://tinyurl.com/rprp-instances>, one can find all of the proposed instances as well as the source codes for reading them and generating new ones. It also has all of the source codes for the devised algorithms and the tables reporting their attained bounds.

5.2. Parameters adjustment and implementation details

All experiments were performed on an Intel® Xeon™ CPU E5-2687 W v3 @ 3.10 GHz computer with 160 GB of RAM and running Ubuntu Linux 18.04. The matheuristics were coded in C++ and the Concert Technology API CPLEX was used to solve production planning problems. The PI_v had at maximum 600 s (s) or 5% estimated gap to run. We solved each instance with the matheuristic algorithms ten times as planned by a power of sample test with a power of 95% to detect the differences in the averages. We set the stopping criterion of 7200 s for all algorithms.

The TILS variant of Algorithm 3 only works with the PP operator. The inner loop (lines 6–19) was modified to repeat at most $\maxIterTILS$ number of times. Every time an improvement is obtained in line 13, the counter $nIter \in [0, \maxIterTILS]$ is reset to 0; otherwise, it is incremented by one.

The parameters α , $\maxWeight$, $\maxIter$, $\maxPertPP$, $\maxPertRD$, $\maxPertLX$, and $\maxPertKK$ were all calibrated using the irace package of [López-Ibáñez, Dubois-Lacoste, Cáceres, Birattari, and Stützle \(2016\)](#), an automatic configuration tool. The parameter α of Algorithm 3 allows the method to explore more configurations without straying too far from the best current solution, as done in the skewed VNS ([Hansen & Mladenović, 2001](#)). The parameter $\maxWeight$ adds an aleatory component to the problem as it modifies the Δ vector of costs. The parameters $\maxIter$ define the maximum number of iterations of the loop between lines 2 and 20 in Algorithm 3. Parameter $\maxPertPP$ bounds the maximum number of moves that perturbation operator PP realizes. The parameters $\maxPertRD$, $\maxPertLX$, $\maxPertKK$ limit the number of times that the routing perturbation operators RD, LX, and KK are applied over the routing solution. Each algorithm was executed thousand times, and the irace picked the best values for each parameter (Table 5).

5.3. Impacts of back-ordering and a heterogeneous fleet

As shown in the literature review, please see Table 1; back-order and heterogeneous fleets are not features commonly adopted in PRP studies. Given that a regular PRP already packs a large number of complicating assumptions, the presence of additional features hinders not only the solution of a test instance but also the development of

Table 5
Adopted parameter values by the devised methods.

Parameter	TILS	OILS	IILS
α	10%	20%	15%
$\maxWeight$	5	5	5
$\maxIter$	300	300	300
$\maxPertPP$	15	–	5
$\maxPertRD$	–	7	7
$\maxPertLX$	–	3	3
$\maxPertKK$	–	15	10

algorithms. Nevertheless, their presence bridges the applicability gap of such solution frameworks.

In industrial applications, back-orders represent a risk of losing sales or even some patrons, but when judiciously allowed, they pose to reduce delivery or inventory costs. According to [Brahimi and Aouam \(2016\)](#), even in a vendor-management inventory policy, back-orders can occur. Some causes can be listed as limited production or storage capacity at the plant facility, prioritization of other products, occurrence of delay in replenishment owing to the fleet capacity, or limited storage capacity at the customers. However, determining the quantities that can be back-ordered without negatively impacting costs or substantially affecting the current production plan is a non-trivial task. Moreover, delivery fleets can be owned or outsourced and may not always be homogeneous, differing in capacity, variable and fixed costs, speeds, and the customers that they can access ([Toth & Vigo, 2014](#)).

To illustrate how these aspects influence the obtained solutions and their costs, using the CPLEX, we solved the proposed set of instances (see Section 5.1) considering three alternative cases: (a) no back-ordering and heterogeneous fleet, (b) neither back-ordering nor heterogeneous fleet, and (c) back-ordering and a homogeneous fleet. These alternative cases are compared with the proposed RPRP, case (d), considering simultaneous back-ordering and a heterogeneous fleet. Some metrics are presented in Table 6 for further discussion.

Table 6 presents the percentage difference among the aforementioned cases (a), (b), and (c) when compared with the proposed RPRP, represented by case (d). A negative value means that for alternative cases, a worse value was found. For each of analyzed cases (a), (b), (c), and (d), when solving the proposed set of instances, the CPLEX was capable of attaining feasible solutions for 74, 70, 84, and 84, respectively. First, it suggests that allowing for back-ordering the demands, cases (c) and (d), eases the resolution of the problems and leads to better costs and use of resources.

For all tested problems and cases, the same production costs were obtained once the production, even if back-ordered, must satisfy customers' demands. When compared with the proposed RPRP, all alternative cases had the worst total cost. The setup costs show that

Table 6
Solutions' comparison among the cases (a), (b) and (c) with (d).

Cases comparison (%)			
Metrics	(a)	(b)	(c)
Total cost	-21.22	-6.88	-7.06
Setup cost	-52.18	-20.07	-12.67
Production costs	0.00	0.00	0.00
Holding costs	1.81	-25.70	-45.06
Back-ordering costs	0.00	0.00	-2,942.89
Fleet cost	-6.75	-0.85	0.40
Traveling cost	-16.68	-8.56	-2.32
#Setups	-50.45	-18.89	-12.89
#Activated vehicles	-5.49	0.00	-3.11
#Visited customers	-4.56	-3.42	-0.36

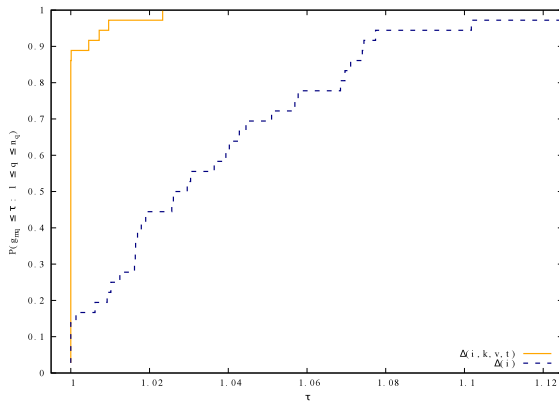


Fig. 2. Comparison of the new Δ_{ik}^{vt} with the Δ_i of Qiu, Wang, Xu et al. (2018).

the alternative scenarios performed much worse than the RPRP case, especially without the back-order policy. (a) is approximately 52% larger than the RPRP. In contrast, this was the sole case that attained slightly better holding costs.

Observing the back-ordering cost metric, case (c) was almost 3,000% worse than our proposed method. As we can see, the absence of both back-ordering and heterogeneous fleet incurs higher fleet and traveling costs, as well as the total cost, for all three alternative cases.

Furthermore, the disallowance of back-order policy and the adoption of a homogeneous fleet can lead to longer travel distances, a larger number of customers visited per period, an increase in the number of vehicles running, and times that the setups occur. These settings raise environmental concerns, as highlighted by Fang et al. (2017), Qiu et al. (2017) and Qiu, Ni et al. (2018).

5.4. Importance of Δ

Recall that our parameter Δ_{ik}^{vt} estimates the impact of routing and distribution decisions when creating a new production plan. It helps to guide CPLEX to obtain more promising production and inventory plans. Here, we compare our parameter with the one introduced in Qiu, Wang, Xu et al. (2018). To the best of our knowledge, this is the first time indirect costs related to delivery variables q are used.

Using both theirs and our parameter Δ , all the three outlined ILS algorithms were tested on 12 instances PX0n5p15tYv (where $X = 2, 3, 4, 5$, and $Y \in \mathcal{V}$). Fig. 2 brings the attained benchmark profile (Dolan & Moré, 2002), where the plots only use the overall best solutions of the algorithms with respect to the adopted parameter Δ . Our adaptive indirect cost Δ_{ik}^{vt} (solid and orange lines) reaches 90% of the best solution values, demonstrating its superiority over Δ_i (dashed and blue lines).

To draw further insights into the contribution of Δ , we performed statistical tests (Table 7). Here, owing to the smaller sample size and the

Table 7

Statistical tests for the comparison of the new Δ_{ik}^{vt} with Δ_i of Qiu, Wang, Xu et al. (2018).

(a) Kolmogorov–Smirnov tests.			(b) Wilcoxon tests.		
Δ	Statistic	p-value	Alternative	Statistic	p-value
Δ_{ik}^{vt}	1.0	0.0	default	36.0	3.1×10^{-6}
Δ_i	1.0	0.0	greater	630.0	1.5×10^{-6}

absence of normality, please see the results of the Kolmogorov–Smirnov tests of Table 7a, which shows that the p -value is less than 0.05, the non-parametric Wilcoxon test (WX) was applied. The alternative hypothesis of the WX signed-rank test verifies whether the results are statistically different; that is, it verifies if the Δ_{ik}^{vt} results are better than the Δ_i results. Table 7b shows the p -values of the Wilcoxon tests and that the results are indeed significantly different besides showing a clear superior performance of Δ_{ik}^{vt} over Δ_i because both p -values tend to zero.

5.5. Comparing the algorithms

We have assessed the performance of the algorithms with respect to the solutions and the time spent to reach them. First, the best solution values attained by the algorithms were compared to the upper bounds of the VINDX formulation. Hereafter, the three devised algorithms – TILS, OILS, and IILS – are compared.

We have generated instances by varying the values of our four attributes: the number of customers (n), the number of products (p), the number of periods (t), and the number of vehicles (v). We have labeled the instances according to these attributes. Usually, for problems with fewer attributes, e.g., the numbers of clients and candidate facilities of an uncapacitated facility location problem, it is possible to correlate the difficulty of solving an instance with the values of these attributes, i.e., with the instance size. Larger values usually imply a more difficult instance to solve. Nonetheless, as in the present problem, when one has more attributes, this correlation may not be direct. As we will show next, depending on how the instance data affect each other, smaller instances can be as difficult to solve as larger ones.

We have provided a *a posteriori* analysis of the instances to propose a typology for them. The idea is to classify the instances according to the attained optimality gaps as a metric. We have used the achieved lower bounds after running CPLEX for six hours and the best-known solutions (BKS) found by the proposed methods to compute the optimality gaps. We have then divided our instances into four *ad hoc* main groups: (1st) Instances that had optimality gaps less than 20%, (2nd) instances that had optimality gaps between 20% and 30%, (3rd) instances that had optimality gaps between 30% and 40%, and (4th) instances that had optimality gaps larger than 40%.

It is worth highlighting that CPLEX has been able to provide an upper bound to 75 of the 108 proposed instances after six hours of computation, when solving the VINDX model, i.e., CPLEX has been unable to find a feasible integer solution for 33 instances or close to 31% of them. On the other hand, all the outlined matheuristics have reached better upper bounds than the CPLEX ones for all instances with an average of fewer than 30 min, for the worst case.

Fig. 3 shows the sorted attained optimality gaps colored by groups. Note that a white space delimits these groups, and the green horizontal line highlights the average optimality gap of 29.35%. Considering the 33 instances that CPLEX has not found an integer feasible solution, 23 of them belong to the third and fourth groups, showing that these groups with larger optimality gaps may be seen as the ones having the more difficult-to-solve instances.

To draw some insights, Table 8 shows a statistical summary of the characteristics of each group. The first (second/third/fourth) group is composed of 12 (44/43/9) instances, having, on average, 31.67 (32.05/36.98/44.44) customers, 4.25 (4.02/4.00/3.56) products, 6.25

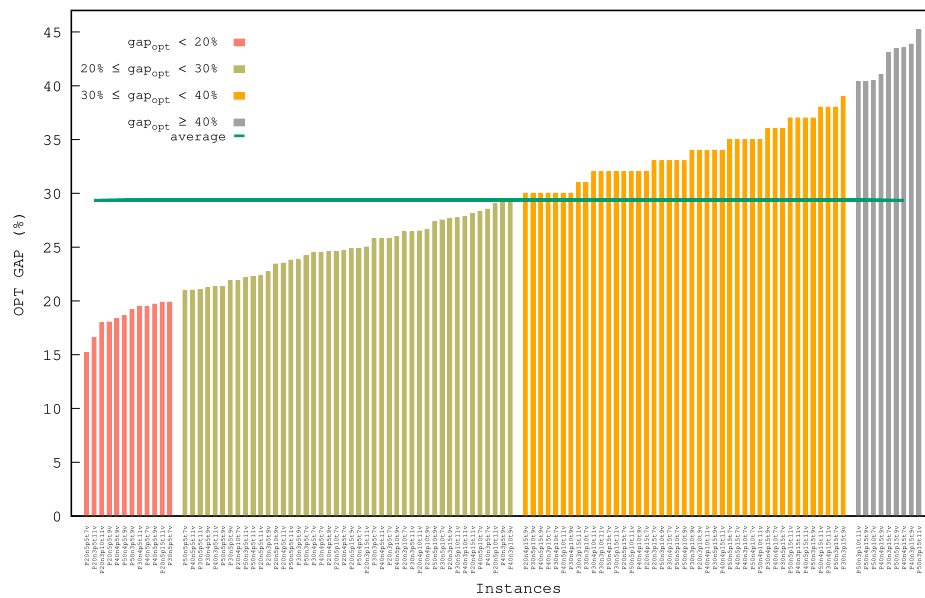


Fig. 3. Attained optimality gaps for the proposed instances.

Table 8

Average attribute value by instance group.

Group	Total	n	p	t	v	OPT(%)
1st	12	31.67	4.25	6.25	9.17	18.52
2nd	44	32.05	4.02	7.50	9.09	24.91
3rd	43	36.98	4.00	12.79	8.91	34.20
4th	9	44.44	3.56	13.89	8.78	42.37

(7.5/12.79/13.89) periods, 9.17 (9.09/8.91/8.78) vehicles, and 18.52% (24.91%/34.20%/42.37%) of optimality gap. Note that the larger the number of clients and periods, the larger the optimality gaps, as one might expect. Handling a large number of clients and periods hinders the solution process.

Nonetheless, on the other hand, in a counter-intuitive manner, the fewer the number of products and available vehicles, the larger the optimality gaps. When more vehicles are available to handle the deliveries, the easier it is to find a feasible solution. One can easily accommodate the delivery orders on the fleet. However, when fewer vehicles are available, one must fine-tune the assessment of the trade-off among the routing cost, the inventory cost, and the back-order cost. It is more difficult to combine mixed loads while adequately serving customers.

When dealing with a smaller number of products, the results show that these instances generally lead to greater gaps. However, this is not an indicative that the problem will be easily solved when a larger number of products is used. One must recall that the problem itself involves other factors such as the arrangement of the demands, the capacity of the inventory level, the size and capacity of the fleet, and the size of the planning horizon. What the results show is that these factors combined are extremely difficult to deal with, explaining somehow why there are just a few works on the literature addressing them.

A different way to visualize the aforementioned insights is by means of a bubble chart as shown in Fig. 4. This figure summarizes the impacts of each of the four attributes in the attained optimality gaps. It is clear that the larger the number of periods (red bubbles) or the number of customers ($n = 40$ or 50), the larger the optimality gaps (vertical axis). However, in a counter-intuitive manner, smaller size fleets, represented by the size of the bubbles, may lead to larger optimality gaps. Instances with fewer products ($p = 3$) have usually

larger optimality gaps. The summary of the upper (lower) bounds, best-known solutions, and optimality gaps can be found at the Appendix A.

To show the difference between the average and minimum objective function values found by the outlined methods and CPLEX, we have used a benchmark profile plot. Performance profiles are graphic tools that aid in evaluating and comparing the performance of a set of algorithms to solve a set of problems over a given performance measure. They have been proposed by Dolan and Moré (2002), wherein more details can be obtained.

The benchmark profile presented in Fig. 5(a) is adopted as a measure of the number of times an algorithm reaches the minimum value found for each instance, also known as the best-known solution (BKS). The OILS algorithm reaches 53.70% of the BKS, followed by the IILS (32.41%), TILS (12.96%), and VINDX (0.93%). In general, the VINDX results were between an average distance from the BKS of approximately 47%, with a maximum of 181% registered.

To conclude this comparative analysis of the objective function values, Fig. 5(b) illustrates the comparison of the averages obtained by the ILS methods in comparison with the best values obtained by the VINDX. OILS has the best average (55.5%), followed by IILS (34.3%), TILS (16.6%), and distance by VINDX (0.93%). One can perceive the preponderance of the heuristics in comparison to the complete model used by the solver in finding good solutions in viable computational time for these problems once the proposed methods easily found solutions for all of them within 1200 s.

The behavior of the matheuristics was studied by conducting statistical tests on the objective values and running times. The parametric methodology, ANOVA was chosen, which, even with the premise of normality of the data, works very well even with asymmetric distributions, as long as the sample size is significantly large, which is the case. All following statistical tests were performed using the MINITAB® 19.

The objective function values of the algorithms are studied. The null hypothesis tests if the average objective function value is equal, while the alternative hypothesis tests if they are not, with $\alpha = 0.05$. Equality of variances of the algorithms was assumed for the analysis. As $\alpha < p\text{-value}$ ($0.05 < 0.295$), the null hypothesis is not rejected (Table 9). Thus, there is sufficient statistical evidence to infer that the three algorithms returned the same average value for the objective function.

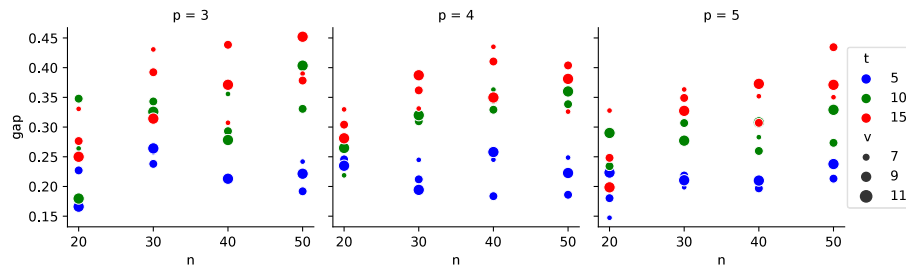


Fig. 4. Comparison of the attained optimality gaps with the problem attributes.

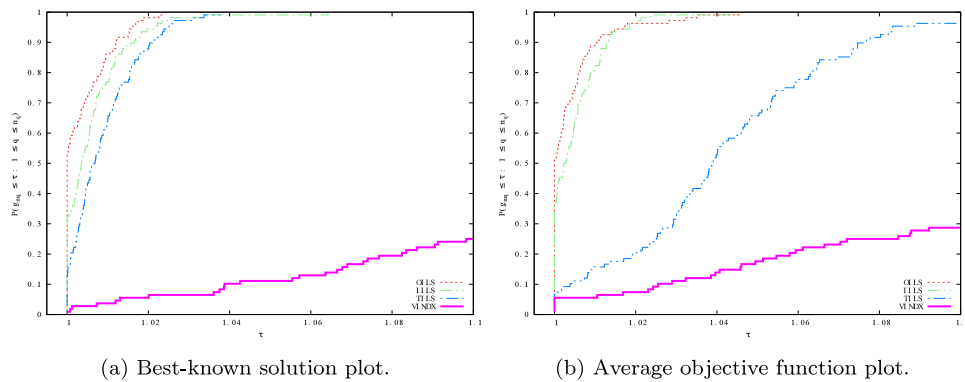


Fig. 5. Benchmark profiles for the devised methods.

Table 9
ANOVA tests for objective function.

Source	F	p-value
o.f.	1.22	0.295

Table 10
Averages and standard deviation for objective function.

Algorithm	N	AVG	SD
TILS	1080	1,031,022	576,228
OILS	1080	995,410	548,640
IILS	1080	1,002,700	551,385

AVG — average, SD — standard deviation.

The Tukey simultaneous tests with 95% confidence for the average objective function was performed. Table 10 lists the averages and standard deviations for the objective function value, while Table 11 presents their relative differences. Once the confidence interval (CI) contains the value 0, the difference between the average objective values can be 0, and the average of the objective function value of these methods is not considerably different.

It is here important to highlight the behavior of the devised methods regarding the attained objective function values. One would expect that the IILS having all the perturbation mechanisms would lead to better solutions than those found by the TILS and OILS, but this was not what happened. Please, bear in mind that when the PP operator changes a production plan, it directly affects the holding and the back-order, and the distribution decisions, which for their turn severely impact the routing decisions. It was then observed that these cascading effects often led to infeasible solutions that needed fixing. Unfortunately, fixing these solutions came at a substantial computational cost, with a further side-effect, the repaired solutions often led to non-promising basins of attraction. Given the aforementioned behavior, one might suggest that the PP operator presented an almost total random behavior. On the other hand, the operational operators, responsible for perturbing the routing decisions, only produced feasible solutions, i.e., solutions

Table 11
Tukey simultaneous tests for differences of objective function averages.

Level Diff	AVG Diff	SE Diff	CI of 95%	T-value	p-value
TILS - OILS	35612	24082	(-20,753; 91,977)	1.48	0.301
IILS - OILS	7291	24082	(-49,074; 63,656)	0.30	0.951
IILS - TILS	-28322	24082	(-84,687; 28,043)	-1.18	0.468

AVG — average, SE — standard error, Diff — difference, CI — confidence interval.

Table 12
Welch tests results.

Source	F	p-value
Opt	14.68	0.00
PI _v	74.72	0.00
VR _t	70.80	0.00
Total	133.04	0.00

that simultaneously respected all of the problem's constraints. Thus, concerning the attained objective function values, the OILS led to better solutions than the IILS and TILS, while the IILS performed in between the OILS and TILS strategies.

The running time was analyzed by considering four different measurements (in seconds) for each algorithm. They are (i) the time until the best solution - Opt; (ii) the cumulative time spent to solve the problems PI_v and (iii) VR_t, and (iv) total running time. The time measure (ii) includes the time spent by the PP, while (iii) accounts for the time spent by the routing operators (RR, SR, KK, LX, and RD).

One-way ANOVA was used to detect any differences between each of these times. The null hypothesis is that if the average time spent is equal and the alternative hypothesis is not, with $\alpha = 0.05$. No equality of variances is assumed for the analysis. The Welch test is significant at the 5% level of significance; that is, there is sufficient statistical evidence to infer that the three algorithms have different Opt, PI_v, VR_t, and total average times (Table 12), once $\alpha \geq p$ -value, and then the null hypothesis is rejected.

On observing Table 13 with the respective standard deviations and averages, we note that the OILS algorithm has the worst average time

Table 13

Standard deviation and average of the times.

Time	TILS		OILS		IILS	
	SD	AVG	SD	AVG	SD	AVG
Opt	654.50	729.60	462.50	621.70	404.98	729.60
PI _v	1198.70	1482.30	547.40	1085.10	369.18	1018.50
VR _t	162.18	126.81	258.16	204.87	256.42	223.40
Total	1082.20	1868.30	456.00	1322.90	219.15	1321.99

SD — standard deviation, AVG — average.

Table 14

Tukey simultaneous tests for differences of time averages.

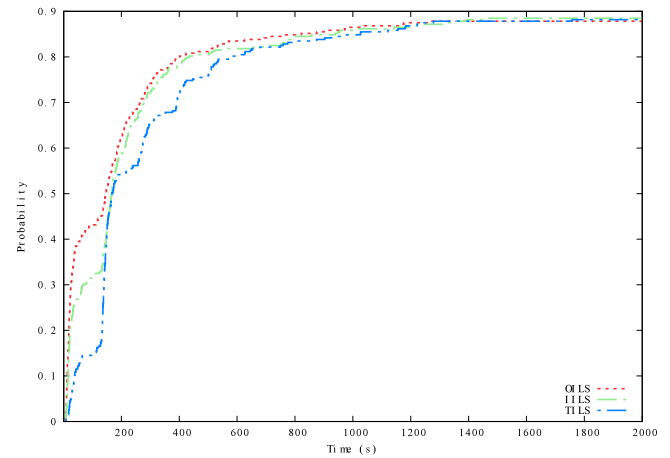
Time	Level Diff	AVG Diff	SE Diff	CI of 95%	T-value	p-value
Opt	TILS - OILS	107.9	24.4	(50.8; 165.0)	4.42	0.000
	IILS - OILS	107.9	24.4	(50.8; 165.0)	4.42	0.000
	IILS - TILS	0.0	28.2	(-65.9; 65.9)	0.00	1.000
PI _v	TILS - OILS	397.3	40.1	(303.4; 491.1)	9.91	0.000
	IILS - OILS	-66.6	20.1	(-113.6; -19.6)	-3.31	0.003
	IILS - TILS	-463.9	38.2	(-553.2; -374.5)	-12.15	0.000
VR _t	TILS - OILS	-78.06	9.28	(-99.78; -56.35)	-8.41	0.000
	IILS - OILS	18.5	11.1	(-7.4; 44.4)	1.67	0.215
	IILS - TILS	96.59	9.23	(74.98; 118.20)	10.46	0.000
Total	TILS - OILS	545.4	35.7	(461.7; 629.0)	15.26	0.000
	IILS - OILS	-0.9	15.4	(-37.0; 35.1)	-0.06	0.998
	IILS - TILS	-546.3	33.6	(-624.9; -467.7)	-16.26	0.000

only for the VR_t when compared with TILS algorithm when compared with the IILS algorithm, they have close total and VR_t times. Finally, when comparing the TILS and IILS algorithms, the average to the best answer (Opt) is identical, and except for the time spent in VR_t, all other times in TILS are worse than IILS. Games-Howell paired comparison tests were performed to infer the average time difference between the algorithms.

The Games-Howell simultaneous test is significant at the 5% level of significance; that is, there is sufficient statistical evidence to infer that the difference in the average times of the three algorithms (Column AVG Diff, Table 14) is different from 0. Subsequently, the null hypothesis regarding the equality of the average times of the three algorithms is rejected ($\alpha \geq p$ -value) in the following pairwise comparisons. For Opt, OILS is different from both TILS and IILS; for PI_v, all algorithms are different among themselves, whereas for VR_t and total average times, TILS is different from both OILS and IILS.

Finally, the performance of the algorithms was examined with a multiple time-to-target (MTTT) plot, an extension of the TTT plots (Aiex, Resende, & Ribeiro, 2002, 2007) and capable of setting multiple instances simultaneously (Reyes & Ribeiro, 2018). For this, the worst objective function values from the five selected instances are defined as their respective targets. These instances were randomly chosen from the group of problems wherein the CPLEX did not obtain any upper bound. Each method was run for 100 executions until the target was found or the limit time was reached. Fig. 6 presents the MTTT graphic, wherein it shows that the OILS algorithm in 80% of attempts, reaches the target within 400 s, overlapping both the IILS and the TILS algorithms, with up to 435 and 580 s, respectively. Once again, the OILS algorithm had the best performance, followed by the IILS and TILS algorithms, in that order.

The analysis is extended to the efficiency of perturbation operators and routing local searches. We consider as efficiency the number of times that the adoption of the operator or local search led to solution improvements. From Table 15, it can be deduced that in the TILS variation, the operator PP is not efficient, leading to only 1.43% improvement. In contrast, when it works together with the routing operators in IILS, its efficiency increases almost 7.5 times. A similar behavior is observed for the routing operators in OILS and IILS; their performance increases by approximately 1% when coupled with PP.

**Fig. 6.** Multiple TTT plot.**Table 15**

Percentage efficiency of the perturbation operators.

	PP	RR	LX	RD	SR	KK
TILS	1.43	—	—	—	—	—
OILS	—	4.93	4.93	4.95	4.84	4.84
IILS	10.72	5.75	5.84	5.8	5.83	5.78

Considering the routing local searches, the VLNS showed itself to be extremely powerful, being responsible for reaching approximately 39.1% of all improvements, followed by the SWAP(2,1), SWAP(2,2), SHIFT(2,0), K-SHIFT(3), K-SHIFT(4), CROSS, and K-SHIFT(5) with 24.2%, 15.1%, 12.3%, 3.7%, 2.1%, 2.0%, and 1.4% of the time, respectively. This performance obtained by the VLNS is due to its well-defined network-flow structure, which enables many more exchanges than the other searches. The set of intra-route searches had improved movements with 2-opt (25.8%), 1-point-move (20.7%), 2-point-move (20.3%), 3-point-move (17.0%), Or-opt (Or = 3, 7.4%; Or = 4, 5.3%; Or = 5, 3.5%), respectively.

After these analyses, it is clear that operational decisions can strongly affect the performance of integrated problems. This corroborates the thesis that by spending more time in the prospection of the routing component, most solutions converge more quickly to better results than those found focused first on the production-inventory problem. This was shown in the performance of both the OILS and IILS algorithms that used operational-level operators. This leads to a reflection on whether decision-making can pay more attention to operational decisions and how they influence tactical decisions.

5.5.1. Benchmark instances

We developed tests over a set of instances proposed by Boudia et al. (2007) and Archetti et al. (2011). The obtained results were compared with those of Russell (2017), Solyali and Sural (2017), and Schenekem-berg et al. (2021). Each algorithm was executed five times for each instance, and a brief analysis is presented as follows:

The first sets, B1, B2, and B3 from Boudia et al. (2007), totaled 90 instances. The algorithms had averages above 17% of the BKS for all sets, demonstrating a very low adherence to these instances. The sets A2 (A50) and A3 (A100) from Archetti et al. (2011), totaled 192 instances with multiple vehicles. For the A2 (A3) set, the results presented a better performance, with an average 4.79% (2.24%) worse than the BKS, with the best results remaining 0.4% (0.3%) from the BKS. Future studies will involve a better calibration of the algorithm parameters to obtain more refined solutions.

6. Conclusions

Owing to the high-level computational challenge offered by the proposed rich production-routing problem, we herein developed approaches that split the problem into a top-down decision manner. A top-tier problem solves the production, inventory, and distribution plans with CPLEX, providing an estimate of the loads to be distributed. The bottom tier heuristically routed the operational level for each period.

Three algorithms embedded in an ILS framework were devised. The first provoked perturbations at the tactical level, modifying the PP and directly affecting the inventory, back-order, and distribution variables. The second explored a set of mechanisms to modify the routing plan, changing the operational level. The last algorithm works by integrating perturbations at both levels.

An important and common point in the implementation of the algorithms was the development of an intuitive method for calculating the costs of the delivered loads. This is because if there is a need to deliver a lot, this considerably affects the costs related to production, inventories, and transportation. In this manner, during each iteration of the method, these indirect costs are updated with the latest information from the solution. Combined with the perturbation mechanisms, it was able to lead to better solutions than those obtained by the commercial solver.

Through computational experiments, the methods were compared with the proposed formulation and among themselves. The CPLEX had difficulty obtaining a solution to most problems within the available time. On the contrary, for those instances wherein a solution was obtained, CPLEX was surpassed, both in running time and objective function values by the three proposed methods.

The results were statistically analyzed, showing the importance of our indirect cost and perturbation mechanisms, especially those that modified route planning. The operational perturbation operator was capable of reaching most of the BKSs with less computational effort for the tested instances.

For future research, it may be interesting, though more challenging, to add more features to PRP. Features such as allowing partial deliveries or lost sales, perishable products, adoption of periodic time windows with demands being served within a given range of periods, heterogeneous and multiple fleets, third-party logistic operators, and proportional service time, may bring theory even closer to practice, which will most likely lead to even greater cost savings.

CRedit authorship contribution statement

Alexandre Fortes: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. **Ricardo Camargo:** Conceptualization, Supervision, Writing – review & editing. **Leandro Reis Muniz:** Conceptualization, Formal analysis. **Fátima Machado de Souza Lima:** Conceptualization, Reviewing and editing. **Fernanda dos Reis Cota:** Conceptualization, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors thank CNPq for financially supporting this research. The second author is funded by the grant #308852/2019-2.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cie.2022.108369>.

References

- Absi, N., Archetti, C., Dauzère-Pérès, S., & Feillet, D. (2014). A two-phase iterative heuristic approach for the production routing problem. *Transportation Science*, 49(4), 784–795.
- Absi, N., Archetti, C., Dauzère-Pérès, S., Feillet, D., & Speranza, M. G. (2018). Comparing sequential and integrated approaches for the production routing problem. *European Journal of Operational Research*, [ISSN: 0377-2217] 269(2), 633–646.
- Adulyasak, Y., Cordeau, J.-F., Jans, R., et al. (2014a). Formulations and branch-and-cut algorithms for multivehicle production and inventory routing problems. *INFORMS Journal on Computing*, 26(1), 103–120.
- Adulyasak, Y., Cordeau, J.-F., Jans, R., et al. (2014b). Optimization-based adaptive large neighborhood search for the production routing problem. *Transportation Science*, 48(1), 20–45.
- Adulyasak, Y., Cordeau, J.-F., Jans, R., et al. (2015a). Benders decomposition for production routing under demand uncertainty. *Operations Research*, 63(4), 851–867.
- Adulyasak, Y., Cordeau, J.-F., Jans, R., et al. (2015b). The production routing problem: A review of formulations and solution algorithms. *Computers & Operations Research*, 55, 141–152.
- Ahuja, R. K. (2017). *Network flows: theory, algorithms, and applications*. Pearson Education.
- Ahuja, R. K., Orlin, J. B., Sharma, D., et al. (1998). New neighborhood search structures for the capacitated minimum spanning tree problem.
- Ahuja, R. K., Orlin, J. B., Sharma, D., et al. (2000). Very large-scale neighborhood search. *International Transactions in Operational Research*, 7(4–5), 301–317.
- Aiex, R. M., Resende, M. G., & Ribeiro, C. C. (2002). Probability distribution of solution time in GRASP: An experimental investigation. *Journal of Heuristics*, 8(3), 343–373.
- Aiex, R. M., Resende, M. G., & Ribeiro, C. C. (2007). TTT plots: a perl program to create time-to-target plots. *Optimization Letters*, 1(4), 355–366.
- Archetti, C., Bertazzi, L., Laporte, G., & Speranza, M. G. (2007). A branch-and-cut algorithm for a vendor-managed inventory-routing problem. *Transportation Science*, 41(3), 382–391.
- Archetti, C., Bertazzi, L., Paletta, G., Speranza, M. G., et al. (2011). Analysis of the maximum level policy in a production-distribution system. *Computers & Operations Research*, 38(12), 1731–1746.
- Archetti, C., & Speranza, M. G. (2016). The inventory routing problem: the value of integration. *International Transactions in Operational Research*, 23(3), 393–407.
- Armentano, V. A., Shiguemoto, A. L., Løkketangen, A., et al. (2011). Tabu search with path relinking for an integrated production-distribution problem. *Computers & Operations Research*, 38(8), 1199–1209.
- Avci, M., & Yildiz, S. T. (2019). A matheuristic solution approach for the production routing problem with visit spacing policy. *European Journal of Operational Research*, 279(2), 572–588.
- Avci, M., & Yildiz, S. T. (2020). A mathematical programming-based heuristic for the production routing problem with transshipments. *Computers & Operations Research*, 123, Article 105042.
- Bard, J. F., & Nananukul, N. (2010). A branch-and-price algorithm for an integrated production and inventory routing problem. *Computers & Operations Research*, 37(12), 2202–2217.
- Bayley, T., & Bookbinder, J. H. (2019). A branch-and-cut algorithm for the production-routing problem with coordinated and capacitated lot-sizes and backorders. In *International workshop on lot-sizing-iwls'2019* (p. 26).
- Belo-Filho, M., Amorim, P., Almada-Lobo, B., et al. (2015). An adaptive large neighbourhood search for the operational integrated production and distribution problem of perishable products. *International Journal of Production Research*, 53(20), 6040–6058.
- Benders, J. F. (1962). Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4(1), 238–252.
- Bianchessi, N., Mansini, R., Speranza, M. G., et al. (2018). A branch-and-cut algorithm for the Team Orienteering Problem. *International Transactions in Operational Research*, 25(2), 627–635.
- Bitran, G. R., & Tirupati, D. (1993). Hierarchical production planning. *Handbooks in Operations Research and Management Science*, 4, 523–568.
- Boudia, M., Louly, M. A. O., Prins, C., et al. (2007). A reactive GRASP and path relinking for a combined production-distribution problem. *Computers & Operations Research*, 34(11), 3402–3419.
- Brahimi, N., & Aouam, T. (2016). Multi-item production routing problem with back-ordering: a MILP approach. *International Journal of Production Research*, 54(4), 1076–1093.
- Brown, G., Keegan, J., Vigus, B., & Wood, K. (2001). The Kellogg company optimizes production, inventory, and distribution. *Interfaces*, 31(6), 1–15.
- Caserta, M., & Voß, S. (2009). Metaheuristics: intelligent problem solving. In *Matheuristics* (pp. 1–38). Springer.
- Çetinkaya, S., Üster, H., Easwaran, G., & Keskin, B. B. (2009). An integrated outbound logistics model for Frito-Lay: Coordinating aggregate-level production and distribution decisions. *Interfaces*, 39(5), 460–475.
- Chandra, P. (1993). A dynamic distribution model with warehouse and customer replenishment requirements. *Journal of the Operational Research Society*, 681–692.

- Chitsaz, M., Cordeau, J.-F., & Jans, R. (2019). A unified decomposition matheuristic for assembly, production, and inventory routing. *INFORMS Journal on Computing*, 31(1), 134–152.
- Clarke, G. u., & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4), 568–581.
- Coelho, L. C., Cordeau, J.-F., Laporte, G., et al. (2013). Thirty years of inventory routing. *Transportation Science*, 48(1), 1–19.
- Coelho, L. C., & Laporte, G. (2013). A branch-and-cut algorithm for the multi-product multi-vehicle inventory-routing problem. *International Journal of Production Research*, 51(23–24), 7156–7169.
- Côté, J., Guastaroba, G., & Speranza, M. (2017). The value of integrating loading and routing. *European Journal of Operational Research*, [ISSN: 0377-2217] 257(1), 89–105.
- Darvish, M., Archetti, C., & Coelho, L. C. (2018). Trade-offs between environmental and economic performance in production and inventory-routing problems. *International Journal of Production Economics*.
- Dayarian, I., & Desautels, G. (2019). A branch-price-and-cut algorithm for a production-routing problem with short-life-span products. *Transportation Science*, 53(3), 829–849.
- Díaz-Madroño, M., Peidro, D., Mula, J., et al. (2015). A review of tactical optimization models for integrated production and transport routing planning decisions. *Computers & Industrial Engineering*, 88, 518–535.
- Dolan, E. D., & Moré, J. J. (2002). Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2), 201–213.
- Fang, X., Du, Y., Qiu, Y., et al. (2017). Reducing carbon emissions in a closed-loop production routing problem with simultaneous pickups and deliveries under carbon cap-and-trade. *Sustainability*, 9(12), 2198.
- Golsefid, A. H., & Jokar, M. R. A. (2020). A robust optimization approach for the production-inventory-routing problem with simultaneous pickup and delivery. *Computers & Industrial Engineering*, Article 106388.
- Groër, C., Golden, B., Wazil, E., et al. (2010). A library of local search heuristics for the vehicle routing problem. *Mathematical Programming Computation*, 2(2), 79–101.
- Hansen, P., & Mladenović, N. (2001). Variable neighborhood search: Principles and applications. *European Journal of Operational Research*, 130(3), 449–467.
- Hansen, P., Mladenović, N., Todosijević, R., & Hanafi, S. (2017). Variable neighborhood search: basics and variants. *EURO Journal on Computational Optimization*, 5(3), 423–454.
- Kytöjoki, J., Nuortio, T., Bräysy, O., & Gendreau, M. (2007). An efficient variable neighborhood search heuristic for very large scale vehicle routing problems. *Computers & Operations Research*, 34(9), 2743–2757.
- Li, Y., Chu, F., Chu, C., & Zhu, Z. (2019). An efficient three-level heuristic for the large-scaled multi-product production routing problem with outsourcing. *European Journal of Operational Research*, 272(3), 914–927.
- Li, Y., Chu, F., Côté, J.-F., Coelho, L. C., & Chu, C. (2020). The multi-plant perishable food production routing with packaging consideration. *International Journal of Production Economics*, 221, Article 107472.
- López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L. P., Birattari, M., & Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3, 43–58.
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2003). Iterated local search. In *Handbook of metaheuristics* (pp. 320–353). Springer.
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2019). Iterated local search: Framework and applications. In *Handbook of metaheuristics* (pp. 129–168). Springer.
- Lysgaard, J. (1997). Clarke & wright's savings algorithm. *Department of Management Science and Logistics, the Aarhus School of Business*.
- Miranda, P. L., Morabito, R., Ferreira, D., et al. (2018). Optimization model for a production, inventory, distribution and routing problem in small furniture companies. *TOP*, 26(1), 30–67.
- Neves-Moreira, F., Almada-Lobo, B., Cordeau, J.-F., Guimarães, L., & Jans, R. (2019). Solving a large multi-product production-routing problem with delivery time windows. *Omega*, 86, 154–172.
- Penna, P. H. V., Subramanian, A., & Ochi, L. S. (2013). An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. *Journal of Heuristics*, 19(2), 201–232.
- Pisinger, D., & Ropke, S. (2010). Large neighborhood search. In M. Gendreau, & J.-Y. Potvin (Eds.), *International Series in Operations Research & Management Science, Handbook of metaheuristics* (146), (pp. 399–420). Springer New York Dordrecht Heidelberg London: Springer.
- Pochet, Y., & Wolsey, L. A. (2006). *Production planning by mixed integer programming*. Springer Science & Business Media.
- Qiu, Y., Ni, M., Wang, L., Li, Q., Fang, X., Pardalos, P. M., et al. (2018). Production routing problems with reverse logistics and remanufacturing. *Transportation Research Part E: Logistics and Transportation Review*, 111, 87–100.
- Qiu, Y., Qiao, J., Pardalos, P. M., et al. (2017). A branch-and-price algorithm for production routing problems with carbon cap-and-trade. *Omega*, 68, 49–61.
- Qiu, Y., Wang, L., Fang, X., Pardalos, P. M., Goldengorin, B., et al. (2018). Formulations and branch-and-cut algorithms for production routing problems with time windows. *Transportmetrica A: Transport Science*, 1–22.
- Qiu, Y., Wang, L., Xu, X., Fang, X., Pardalos, P. M., et al. (2018). A variable neighborhood search heuristic algorithm for production routing problems. *Applied Soft Computing*, 66, 311–318.
- Reimann, M., Tavares Neto, R., Bogendorfer, E., et al. (2014). Joint optimization of production planning and vehicle routing problems: A review of existing strategies. *Pesquisa Operacional*, 34(2), 189–214.
- Reyes, A., & Ribeiro, C. C. (2018). Extending time-to-target plots to multiple instances. *International Transactions in Operational Research*, 25(5), 1515–1536.
- Russell, R. A. (2017). Mathematical programming heuristics for the production routing problem. *International Journal of Production Economics*, 193, 40–49.
- Schenekemberg, C. M., Scarpin, C. T., Pecora Jr, J. E., Guimarães, T. A., & Coelho, L. C. (2021). The two-echelon production-routing problem. *European Journal of Operational Research*, 288(2), 436–449.
- Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In *International conference on principles and practice of constraint programming* (pp. 417–431). Springer.
- Solyali, O., & Süral, H. (2017). A multi-phase heuristic for the production routing problem. *Computers & Operations Research*, 87, 114–124.
- Souza, M. J., Coelho, I. M., Ribas, S., Santos, H. G., & Merschmann, L. H. d. C. (2010). A hybrid heuristic algorithm for the open-pit-mining operational planning problem. *European Journal of Operational Research*, 207(2), 1041–1051.
- Stålhane, M., Andersson, H., Christiansen, M., & Fagerholt, K. (2014). Vendor managed inventory in tramp shipping. *Omega*, [ISSN: 0305-0483] 47, 60–72.
- Thompson, P. M., & Orlin, J. B. (1989). The theory of cyclic transfers. In *OPERATIONS RESEARCH CENTER WORKING PAPER, MIT*.
- Toth, P., & Vigo, D. (2014). *Vehicle routing: problems, methods, and applications*. SIAM.
- Watanabe, H., Cirino, R., Soler, W., Santos, M., et al. (2017). Solution methods for the integrated production routing problem. *Salesian Journal on Information Systems*, 1(19), 53–61.